AJ Pilot

PROJECT DOCUMENTATION

# Autopilot for your servers

Business, product, architecture, data, API, security and operations — in one book.


01 Introduction

02 Business Requirements

03 Product Requirements

04 System Design

05 Data Model

06 Database Design

07 API Reference

08 Security Design

09 Deployment & Operations

10 Limitations & Roadmap



| VERSION | DATE | STATUS | SOURCE |
| --- | --- | --- | --- |
| 0.1.0 | 29 October 2026 | Living document | docs/ folder of the aj-pilot repository |

# How to use this document

This book brings together every project document for **AJ Pilot**, the self-hosted control panel that deploys apps, manages Ubuntu servers and connects domains over SSH, sold to organisations as plans paid through Razorpay. Each chapter stands on its own, so read the ones for your role first.

| If you are… | Start with | Because it covers |
|---|---|---|
| **Founder / business owner** | 01 Introduction · 02 Business Requirements · 10 Limitations & Roadmap | The problem, the market, how the product earns money, and the risks. |
| **Product manager / QA** | 03 Product Requirements · 02 Business Requirements · 10 Limitations & Roadmap | Every requirement with an ID, the user journeys, and what is not built yet. |
| **New developer** | 01 Introduction · 04 System Design · 05 Data Model · 07 API Reference | Architecture first, then data, then the API you will be changing. |
| **DevOps / platform owner** | 09 Deployment & Operations · 08 Security Design · 06 Database Design | How to run it in production, harden it, back it up and fix it. |
| **Security reviewer** | 08 Security Design · 04 System Design · 07 API Reference | Threat model, request pipeline and permission rules. |

### Conventions

- **IDs** such as `BR-05` (business rule) and `FR-B4` (functional requirement) can be quoted in tickets and test cases.
- **Diagrams** show flows left to right or top to bottom; sequence diagrams read top to bottom in time.
- **Permissions** are `view`, `create`, `edit`, `delete`, `members` and `orgs` (Chapter 03, section 1).
- **File paths** in grey brackets point into the source code, for example (src/lib/razorpay.js).
- Statements marked *proposed* or *recommendation* are not yet built.

# Contents


**01 Introduction** 4

What AJ Pilot is, the document set, and the product at a glance

At a glance · Keeping these documents current

**02 Business Requirements** 6

Why the product exists, who pays, how it makes money

Executive summary · Problem statement · Vision and objectives · Target market and personas · Scope · Business model · Business rules · Success metrics (proposed) · Stakeholders · Assumptions and constraints · Business risks

**03 Product Requirements** 11

What every module must do, roles, user journeys

Roles and permissions · Modules and functional requirements · Key user journeys · Non-functional requirements

**04 System Design** 17

Architecture, components, request pipeline, key flows

Context · Technology choices · Source layout · Request pipeline · Executing work on a server · Key flows · Multi-tenancy · Front-end architecture · Background and in-process state · Error handling conventions

**05 Data Model** 25

Entities, relationships, tenancy and encryption

Entity-relationship overview · Tables by domain · Integrity and deletion · Data that is not in the database · Backup guidance

**06 Database Design** 29

Every table, column, key and index in detail

Engine and conventions · Schema lifecycle · Entity-relationship diagram (physical) · Table definitions · JSON column shapes · Referential integrity · Multi-tenancy rules · Important query patterns · Retention and growth · Recommended improvements · Backup and restore

**07 API Reference** 46

Every endpoint, with the permission it needs

Conventions · Endpoints outside the routers · Endpoints by module

**08 Security Design** 58

Threats and the controls against each

Assets · Threat model and controls · Cryptography · Trust boundaries · Hardening checklist for production · Known gaps and recommendations

**09 Deployment & Operations** 62

Install, configure, run, back up, troubleshoot

Requirements · Production topology · Installation · Configuration reference · Operating the panel · Troubleshooting · Development setup

**10 Limitations & Roadmap** 67

Known constraints, technical debt, next steps

Architectural limitations · Product gaps · Engineering health · Suggested roadmap · Decision log (to maintain)

CHAPTER 01

# Introduction

What AJ Pilot is, the document set, and the product at a glance

AJ Pilot is a multi-tenant control panel for Ubuntu servers. It works over SSH, so nothing is installed on the managed servers. It deploys apps from GitHub, GitLab and Bitbucket, runs Docker and databases, puts apps on custom domains with SSL, and charges organisations for plans through Razorpay.

This folder holds the project's business and technical documents. The root README (README.md) remains the hands-on user guide (feature walkthroughs, every setting, the full endpoint table).

| # | Document | For | What it answers |
|---|---|---|---|
| 1 | Business Requirements (BRD) | Founders, product, sales | Why the product exists, who pays, how it makes money, business rules |
| 2 | Product Requirements (PRD) | Product, engineering, QA | What every module must do, roles, key user journeys |
| 3 | System Design (HLD) | Engineering | Architecture, components, request pipeline, main flows |
| 4 | Data Model | Engineering, DBA | Tables, relationships, tenancy and encryption of data |
| 5 | API Reference | Engineering, integrators | Every endpoint by module, middleware and permission rules |
| 6 | Security Design | Engineering, security review | Threat model and the controls against each threat |
| 7 | Deployment & Operations | DevOps, platform owner | Installing, configuring, running and recovering the panel |
| 8 | Limitations & Roadmap | Product, engineering leads | Known constraints, technical debt, recommended next steps |
| 9 | Database Design | Engineering, DBA | Every column, type, key and index; migrations; JSON shapes; retention |

## At a glance

| | |
|---|---|
| **Runtime** | Node.js ≥ 20, Express 4, ES modules |
| **Front end** | Single-page app in plain JavaScript ( `public/` ), no build step |
| **Database** | MySQL 8 (the panel's own data) |
| **Server access** | SSH ( `ssh2` ), optionally relayed through a *connector* on a user's PC |
| **Real-time** | WebSockets ( `ws` ) for connectors and the browser terminal; Server-Sent Events for live stats |
| **Integrations** | GitHub / GitLab / Bitbucket (OAuth or token), Docker Hub, Cloudflare (DNS, Zero Trust tunnels), Let's Encrypt, Razorpay, SMTP |
| **Size** | ~26k lines of server code, 14 route modules, ~260 API endpoints, 25 tables |

## Keeping these documents current

**Rule: documentation first.** Every new feature or behaviour change updates the docs *before* the code, in the same change. That means three sets: the system docs (this folder), the client docs (the root README (README.md)), and the PDF copies in `docs/pdf/`. The full checklist of which change touches which document is in CLAUDE.md.

- Facts in these documents were taken from the code. When code and document disagree, the code is right; fix the document in the same pull request.

- Update the **API Reference** when a route is added or removed. Update the **Data Model** and **Database Design** when a `CREATE TABLE` or a migration in `src/db/index.js` changes.
- Diagrams are written in Mermaid, so GitHub and most editors render them with no extra tooling.

CHAPTER 02

# Business Requirements

Why the product exists, who pays, how it makes money

| | |
|---|---|
| **Product** | AJ Pilot — "Autopilot for your servers" |
| **Document owner** | Product / platform owner |
| **Status** | Living document. It describes the product as built, and marks proposals as proposals |

## 1. Executive summary

Small teams that run their own Linux servers spend a lot of time on manual operations. They SSH into each machine, write ad-hoc deploy scripts, set up nginx and SSL certificates by hand, and handle database credentials and `.env` files with little control. Hosted platforms (PaaS) remove that work but cost several times more than a VPS and take away control of the infrastructure.

AJ Pilot gives these teams a PaaS-like experience on their own VPS servers. A web panel connects to any Ubuntu server over SSH, with no agent to install. From there a team can deploy apps from git, auto-deploy on every push or merge, install databases with one click, and publish apps on their own domains with free SSL. Every organisation's data is kept separate from every other's.

The platform owner runs AJ Pilot as a **multi-tenant SaaS**. Organisations sign up, start a free trial, and move to paid monthly or yearly plans paid through Razorpay. The same codebase can also run as a **private, self-hosted panel** with public sign-up turned off.

## 2. Problem statement

| Pain | Who feels it | Today's workaround |
|---|---|---|
| Deploying means SSH, `git pull`, rebuild and restart, every time | Developers, freelancers | Shell scripts, manual steps, mistakes |
| Domains, nginx and SSL are fiddly and easy to break | Everyone | Copying config snippets, expired certificates |
| Secrets live in `.env` files passed around by chat | Teams, agencies | Shared documents, Slack messages |
| No view of what is running where across many servers | Agencies, IT teams | Spreadsheets, memory |
| Giving a client or junior developer safe access is hard | Agencies, DevOps | Sharing root passwords |
| A PaaS is easy but expensive and locks you in | Startups | Paying the premium, or going back to scripts |

## 3. Vision and objectives

**Vision:** anyone who can rent a VPS can run production apps on it with the convenience of a hosting platform.

| Objective | How the product meets it |
|---|---|
| O1 — Bare server to live app without a terminal | Add server → install Docker → deploy repo → attach domain (five guided steps) |
| O2 — Hands-off releases | Auto deploy on push or on merged pull/merge requests, by webhook or polling |
| O3 — Safe for teams and clients | Organisations, four roles, activity log, encrypted secrets |

| Objective | How the product meets it |
|---|---|
| O4 — Works where servers actually are | Direct SSH, or through a *connector* on a PC that has the VPN |
| O5 — A sustainable business | Free trial, paid plans with usage limits, online payment, lead pipeline |

## 4. Target market and personas

| Persona | Situation | What they value most |
|---|---|---|
| **Agency** | Hosts many clients' apps on its own servers | One organisation per client, client logins with roles, domains and SSL per client |
| **Startup** | Wants "push to main and it is live" without PaaS prices | Auto deploy on merge, staging and production environments, low cost |
| **Freelancer** | Builds and hands over client projects | Fast deploys, domain and SSL done, invite the client as a viewer |
| **IT / DevOps team** | Standardises servers and lets developers serve themselves | Permissions, audit trail, Linux user and SSH key management |

**Supported infrastructure:** any Ubuntu server reachable over SSH, from any provider (DigitalOcean, AWS, Hetzner, Linode, Hostinger and others) or on-premises.

## 5. Scope

### 5.1 In scope (built)

1. **Servers:** add over SSH, system profile, live statistics, health monitoring, reboot, browser terminal.
2. **Server administration:** systemd services, cron jobs, Ubuntu users and SSH keys, file manager, nginx sites and SSL, nginx log analysis with blocklist and fail2ban.
3. **Installations:** one-click Docker, Docker Compose, MySQL, PostgreSQL, MongoDB, Redis, EMQX and Elasticsearch (as containers).
4. **App deployment:** Node.js family (Next.js, NestJS, Nuxt, Angular, React/Vite/Vue, static, plain Node) built into Docker images, with one or many containers, volumes, environment variables and an optional Docker Hub push.
5. **Auto deploy:** by webhook (GitHub, GitLab, Bitbucket) or polling, triggered on every commit or only on merges.
6. **Domains:** Cloudflare DNS + nginx + Let's Encrypt, or a Cloudflare Zero Trust tunnel for servers with no open ports.
7. **Databases:** browse and manage MySQL, PostgreSQL, MongoDB, Redis and Elasticsearch (databases or indices, users, grants, settings, read-only queries).
8. **Integrations:** git accounts (OAuth or token), Docker Hub, Cloudflare, CI runners (GitHub/GitLab).
9. **Environments:** named, encrypted variable sets reused across apps and services.
10. **Organisations and people:** multi-tenancy, four roles, team management, activity log.
11. **Commercial:** public landing page, self sign-up with OTP and email verification, plans, free trial, Razorpay checkout, payment history, super-admin console (organisations, plans, payments, leads CRM).

### 5.2 Out of scope (currently)

- Operating systems other than Ubuntu/Debian-family Linux on managed servers.
- Runtimes other than Node.js for the build-and-deploy pipeline. A repository can still bring its own `Dockerfile`.
- Automatic recurring billing. A paid plan is bought one cycle at a time; see §7.
- Invoicing and tax documents (GST invoices).
- Real SMS delivery for one-time codes. A provider hook exists, but no provider is integrated yet.

# 6. Business model

## 6.1 Revenue

- **Subscriptions per organisation**, billed **monthly or yearly**. The currency is set per plan (INR by default).
- Plans are **created and priced by the super admin** in the platform console. Prices, limits, features, the "most popular" flag, visibility and order are data, not code.
- Plans are differentiated by **usage limits**:

| Limit key | Counts |
|---|---|
| `servers` | servers in the organisation |
| `apps` | deployed apps |
| `users` | team members |
| `databases` | database connections (MySQL, PostgreSQL, MongoDB, Redis, Elasticsearch) |
| `domains` | app domains |

A missing limit means unlimited. A request that would go over a limit is refused with a clear message (HTTP 403 with `limitReached`).

## 6.2 Free trial

- A free plan (price 0) acts as a trial that lasts `FREE_TRIAL_DAYS` (default **30**). Each organisation can use it **once**.
- When the trial ends the panel is **locked, not deleted**. The user can only reach Plan & Billing until they choose a paid plan.
- The trial's progress is visible throughout: a header pill, a banner that changes to a warning tone when 5 or fewer days are left, and a progress bar on the billing page.

## 6.3 Payment

- **Online (Razorpay configured):** choosing a paid plan opens Razorpay Checkout immediately (card, UPI, netbanking, wallet). The plan **activates automatically once the server has verified the payment**, and a success confirmation is shown.
- **Offline (no Razorpay keys):** a paid plan becomes a *request*. The super admin records the payment by hand and switches the plan on.
- A paid plan chosen at sign-up opens Razorpay on the user's first sign-in.
- Every organisation sees its **payment history**: online payments, payments recorded by hand, and failed or abandoned attempts.

## 6.4 Sales pipeline (leads)

- The public "Get started" form creates a **lead**. It records UTM source, referrer, device and approximate location.
- The super admin works leads through `new → contacted → qualified → proposal → won / lost`, adds notes and an expected value, and **converts a won lead into a client organisation** with its first admin and plan.

# 7. Business rules

| ID | Rule |
|---|---|
| BR-01 | Everything an organisation owns (servers, credentials, apps, environments, connectors, payments) is visible only to that organisation, and to the super admin. |

| ID | Rule |
|---|---|
| BR-02 | A client organisation must have an active (or trial / past-due) subscription before anything it owns can be used. Billing stays reachable without one. |
| BR-03 | The free plan can be used once per organisation and lasts `FREE_TRIAL_DAYS`. |
| BR-04 | An ended plan locks the panel; no data is deleted. |
| BR-05 | A paid plan activates only after the payment has been verified **with Razorpay on the server** (signature, captured status, exact amount and currency). The browser's word is never enough. |
| BR-06 | A payment is applied **exactly once**, even if checkout, webhook and reconciliation all report it. |
| BR-07 | A payment whose confirmation was lost (tab closed, popup blocked) is recovered automatically, by webhook or by reconciliation within a minute of the next billing page visit. |
| BR-08 | Only an organisation admin (permission `members`) can choose or pay for a plan. |
| BR-09 | Creating a resource over a plan limit is refused; existing resources keep working. |
| BR-10 | Only the super admin manages plans, organisations, other admins, recorded payments and leads. |
| BR-11 | Self sign-up requires a verified mobile OTP, a verified email link, and acceptance of the current Terms version. |
| BR-12 | `ALLOW_SIGNUP=false` turns the product into a private panel where accounts are created only by admins. |

## 8. Success metrics (proposed)

The platform console already shows payments collected, this month's revenue and lead counts. The KPIs below are recommendations; they are **not yet measured by the product**.

| Area | KPI |
|---|---|
| Acquisition | Leads per month by source; sign-ups per month |
| Activation | % of new organisations that deploy their first app within 7 days of sign-up |
| Conversion | Trial → paid conversion rate; lead → won rate |
| Revenue | MRR / ARR, ARPA (average revenue per organisation), yearly vs monthly mix |
| Retention | Monthly logo churn; renewals after a plan expires |
| Product health | Deploy success rate; median deploy duration; servers offline per day |

## 9. Stakeholders

| Role | Interest |
|---|---|
| Platform owner (super admin) | Revenue, plans, clients, leads, platform health |
| Organisation admin | Their team, their plan and payments, their infrastructure |
| Editor / developer | Deploying and operating apps and servers |
| Viewer (often a client) | Seeing status and logs without being able to change anything |
| Razorpay, Cloudflare, git providers | External services the product depends on |

## 10. Assumptions and constraints

- Customers already have, or can rent, Ubuntu servers, and can supply SSH credentials with root or sudo.
- The panel's own MySQL database and its **encryption key** are the business-critical assets. Losing the key makes every stored credential unreadable (see Operations).
- Razorpay is the only online payment gateway, which suits an India-first market.

- The panel runs as a **single Node.js process** (see Limitations).

## 11. Business risks

| Risk | Impact | Mitigation |
|---|---|---|
| A security incident: the panel holds root access to customer servers | Severe: reputation and liability | Encryption at rest, strict tenancy, role checks on the server, origin checks, host-key pinning; see Security |
| Loss of the encryption key or database | All customer credentials lost | Back up the key separately from the database; restore drills |
| A payment taken but the plan not activated | Support load, refunds | Server-side verification, webhook, automatic reconciliation |
| Fixed OTP left on in production | Account takeover | Set `OTP_MODE=random` and connect an SMS provider before launch |
| Dependence on a single gateway or region | Lost sales outside India | Add a second gateway when expanding (roadmap) |

CHAPTER 03

# Product Requirements

What every module must do, roles, user journeys

This document lists what each module must do. The IDs ( `FR-xx` ) can be referenced from tickets and test cases. The root README (README.md) shows how each feature looks to a user.

## 1. Roles and permissions

| Permission | Super admin | Admin | Editor | Viewer |
|---|---|---|---|---|
| `view` — see everything in the organisation | ✓ | ✓ | ✓ | ✓ |
| `create` — add things (any `POST` ) | ✓ | ✓ | ✓ | — |
| `edit` — change things (any `PUT` ), open the terminal, see secret values | ✓ | ✓ | ✓ | — |
| `delete` — remove things (any `DELETE` ) | ✓ | ✓ | — | — |
| `members` — manage people, choose and pay for the plan | ✓ | ✓ | — | — |
| `orgs` — organisations, plans, platform console | ✓ | — | — | — |

- **FR-R1:** Permissions are enforced by the API on every request. The UI hides controls a role cannot use, but the API is the authority.
- **FR-R2:** Some `POST` endpoints only read (test a connection, collect facts, verify a credential, browse a database). These stay open to viewers through an explicit allowlist.
- **FR-R3:** The only super admin cannot be deleted, demoted or disabled.
- **FR-R4:** Changing a person's role, status or password ends all of that person's sessions immediately.

## 2. Modules and functional requirements

### 2.1 Accounts and sign-in

| ID | Requirement |
|---|---|
| FR-A1 | First run (no accounts): a setup form creates the super admin and the first organisation. `SUPER_ADMIN_*` in `.env` can also enforce the owner account on every boot. |
| FR-A2 | Self sign-up (when `ALLOW_SIGNUP=true` ) collects name, email, mobile, password, company, a plan and a billing cycle, plus acceptance of the Terms. |
| FR-A3 | Sign-up and sign-in both require a 6-digit one-time code (OTP), with resend limits and expiry ( `OTP_MINUTES` ). |
| FR-A4 | A self-signed-up account must click an email verification link before its first sign-in. |
| FR-A5 | Sessions use an HttpOnly cookie and last `AUTH_SESSION_DAYS` (default 14). |
| FR-A6 | Admins and super admins must add a mobile number before they continue. |

### 2.2 Plans, billing and payments

| ID | Requirement |
|---|---|
| FR-B1 | The billing page shows the current plan, trial progress, usage against limits, available plans (monthly/yearly toggle) and payment history. |
| FR-B2 | Choosing a free plan activates it at once (only if the organisation has never used the free trial). |

| ID | Requirement |
|---|---|
| FR-B3 | Choosing a paid plan, with Razorpay configured, **opens Razorpay Checkout directly**. With no keys configured, it creates a request for the super admin. |
| FR-B4 | After checkout the server verifies the signature and the captured payment, then activates the plan and shows a **"Payment successful — your plan is active"** dialog with plan, amount, renewal date and payment ID. |
| FR-B5 | If checkout closes without an answer, the server asks Razorpay whether the payment went through and activates the plan if so. |
| FR-B6 | Razorpay's webhook (`payment.captured`, `order.paid`, `payment.failed`) completes or fails orders independently of the browser. |
| FR-B7 | A paid plan picked at sign-up opens Razorpay automatically on first sign-in, once per request per browser. |
| FR-B8 | Payment history lists paid, failed and incomplete payments with date, plan, method, reference, status and amount. |
| FR-B9 | An ended plan locks every page except Plan & Billing. Nothing is deleted. |
| FR-B10 | Creating a resource over a plan limit is refused with a message naming the limit. |

## 2.3 Servers

| ID | Requirement |
|---|---|
| FR-S1 | Add a server with host, port, username and a password or private key (with optional passphrase and sudo password). Test the connection before saving. |
| FR-S2 | Collect a **system profile** (hardware, storage, network, ports, processes, services, updates, security, tooling, Docker, MySQL) and keep each collection as a snapshot. |
| FR-S3 | Health monitor: a TCP probe of the SSH port every 5 s, plus a real sign-in check every 10 min. Show the result as online/offline and "cannot sign in". |
| FR-S4 | Live view: CPU, memory, network, disk I/O and processes, streamed while the tab is open. A *Statistics* chart on the Overview shows CPU, memory and disk over the last day, week or month, with time spent normal / warning / critical (sampled every 5 min, kept 35 days). |
| FR-S5 | Pin the host key on first use. A changed key blocks the connection until an admin resets it. |
| FR-S6 | Server tabs: Overview, Storage (file manager), Network, Processes, System, Live, Apps, Docker, Nginx, Nginx Logs, Cron, Users, Services, Runners, Terminal. |
| FR-S7 | **Terminal:** an interactive shell in the browser (xterm.js over WebSocket), for roles with `edit`. It closes after 30 min with no input; a user can have at most 5 open. Opening and closing are logged; keystrokes are not. |
| FR-S8 | Reboot the server (`edit`). |
| FR-S9 | **Connectors:** a downloadable program for a user's PC that relays SSH to servers only that PC can reach (behind a VPN). The panel tries direct SSH first and falls back to a connector automatically. |

## 2.4 Server administration

| ID | Requirement |
|---|---|
| FR-SA1 | systemd: list, start/stop/restart/enable units, read the journal, create units. Delete only units the panel created. |
| FR-SA2 | Cron: list jobs and add, edit and remove them in user crontabs. Install cron if it is missing. |
| FR-SA3 | Ubuntu users: add, edit, lock and delete users; set passwords and sudo; add and remove SSH keys; generate key pairs (the private key is shown once and never stored). |
| FR-SA4 | File manager: browse, read, edit, upload (up to 50 MB), download, move, copy, chmod, extract and delete, as root. |
| FR-SA5 | Nginx: install it, test and reload (reload is refused if the config test fails), manage sites and upstreams, issue and renew Let's Encrypt certificates. |
| FR-SA6 | Nginx logs: analyse access and error logs, block IPs, set up fail2ban. |
| FR-SA7 | Docker: containers (inspect, logs, start/stop/remove), networks, volumes, registry login. |

## 2.5 Installations

| ID | Requirement |
|---|---|
| FR-I1 | The catalog has Docker and Docker Compose (installed on the host) and MySQL, PostgreSQL, MongoDB, Redis, EMQX and Elasticsearch (as containers). |
| FR-I2 | The user chooses the port, version, network and credentials. Ports can be changed later. Passwords are encrypted at rest. |
| FR-I3 | Removing an installation keeps its data volume unless the user explicitly asks to delete it. |

## 2.6 Apps (custom services)

| ID | Requirement |
|---|---|
| FR-D1 | Inspect the repository through the git API before cloning. Refuse it if there is no `package.json` or no start command, and say why. |
| FR-D2 | Detect the project type (Next.js, NestJS, Nuxt, Node service, Angular, React/Vite/Vue, static), the Node version, the package manager and the build/start commands. All are editable. |
| FR-D3 | Build a Docker image on the target server. Static front ends are served by nginx with an SPA fallback. A repository's own `Dockerfile` can be used instead. |
| FR-D4 | Run one or several containers on chosen ports, with named volumes (default `<name>-data` at `/app/data`), environment variables, a network and a restart policy. |
| FR-D5 | The previous release keeps running until the new image has built. |
| FR-D6 | Deploys run in the background. The UI shows progress; a second deploy of the same app while one is running is refused; a deploy stuck for 40 minutes is released. |
| FR-D7 | Optionally push the image to Docker Hub. |
| FR-D8 | Per app: redeploy, logs (container and build), environment editor (`edit` only), container set changes, start/stop/restart, remove (volumes kept unless chosen). |
| FR-D9 | **Auto deploy:** by webhook or polling, on every commit or only on merges, for the app's branch. Each run is recorded with its commit/PR, author, status and log. |
| FR-D10 | **Domains:** attach a domain using Cloudflare DNS + nginx + Let's Encrypt, or a Cloudflare tunnel. Retry or remove it. |
| FR-D11 | Git tokens in clone URLs are base64-encoded in transit and scrubbed from every stored or displayed log. |
| FR-D12 | **Usage:** each app card shows the app's CPU, memory and network (refreshed every 20 s while visible). The app's *Live stats* tab streams CPU, memory, network and disk I/O every 5 s for the app and each container (state, uptime, PIDs, restarts, health, OOM kills), and warns when a container is down, restarting, unhealthy or near a limit. The Overview shows the app's CPU and memory history over a day, week or month. |

## 2.7 Databases

| ID | Requirement |
|---|---|
| FR-DB1 | Connect to MySQL, PostgreSQL, MongoDB, Redis and Elasticsearch, directly or tunnelled through a server's SSH. |
| FR-DB2 | Overview statistics; browse databases, tables and collections; manage databases, users, grants and settings. |
| FR-DB3 | The query console is **read-only**. |
| FR-DB4 | Redis: find keys with SCAN (never KEYS), read large values page by page, edit and delete keys. |
| FR-DB5 | Elasticsearch: connect over HTTP or HTTPS with the `elastic` user (or none when security is off). Show cluster health, nodes, heap, disk and shards; list indices (system indices hidden from changes); create an index with shard and replica counts; delete an index after its name is typed back; per index show documents, size, shards, aliases, settings and field mappings; per field show how many documents have it and its top values. Manage security users (create, password, enable / disable, delete) and give them a built-in or custom role, or read / write / full access to an index pattern. Change a short list of persistent cluster settings. Query console takes Dev Tools syntax (`GET /path` plus an optional JSON body) and allows only `GET` and search-type `POST` requests. |

| ID | Requirement |
|---|---|
| FR-DB6 | Installing Elasticsearch from Installations adds its connection under Databases by itself, as the `elastic` user when a password was set. |

## 2.8 Accounts and integrations

| ID | Requirement |
|---|---|
| FR-C1 | Git: connect GitHub, GitLab and Bitbucket by OAuth (with PKCE and token refresh) or by pasting a token. Supports self-hosted GitHub Enterprise and GitLab. |
| FR-C2 | Docker Hub and Cloudflare: guided creation of a personal token, then verify and browse (repositories and tags; zones and DNS records; Zero Trust). |
| FR-C3 | CI runners: register and install GitHub/GitLab self-hosted runners on a server, check their status, control and remove them. |
| FR-C4 | Environments: named, encrypted sets of variables, created by importing a `.env` file, duplicated, and attached to apps and installations. Viewers see key names only. |

## 2.9 Teams and audit

| ID | Requirement |
|---|---|
| FR-T1 | Admins add, edit and remove members of their own organisation and assign roles (never super admin). |
| FR-T2 | The activity log records every change, with the organisation and the user who made it. |

## 2.10 Platform console (super admin)

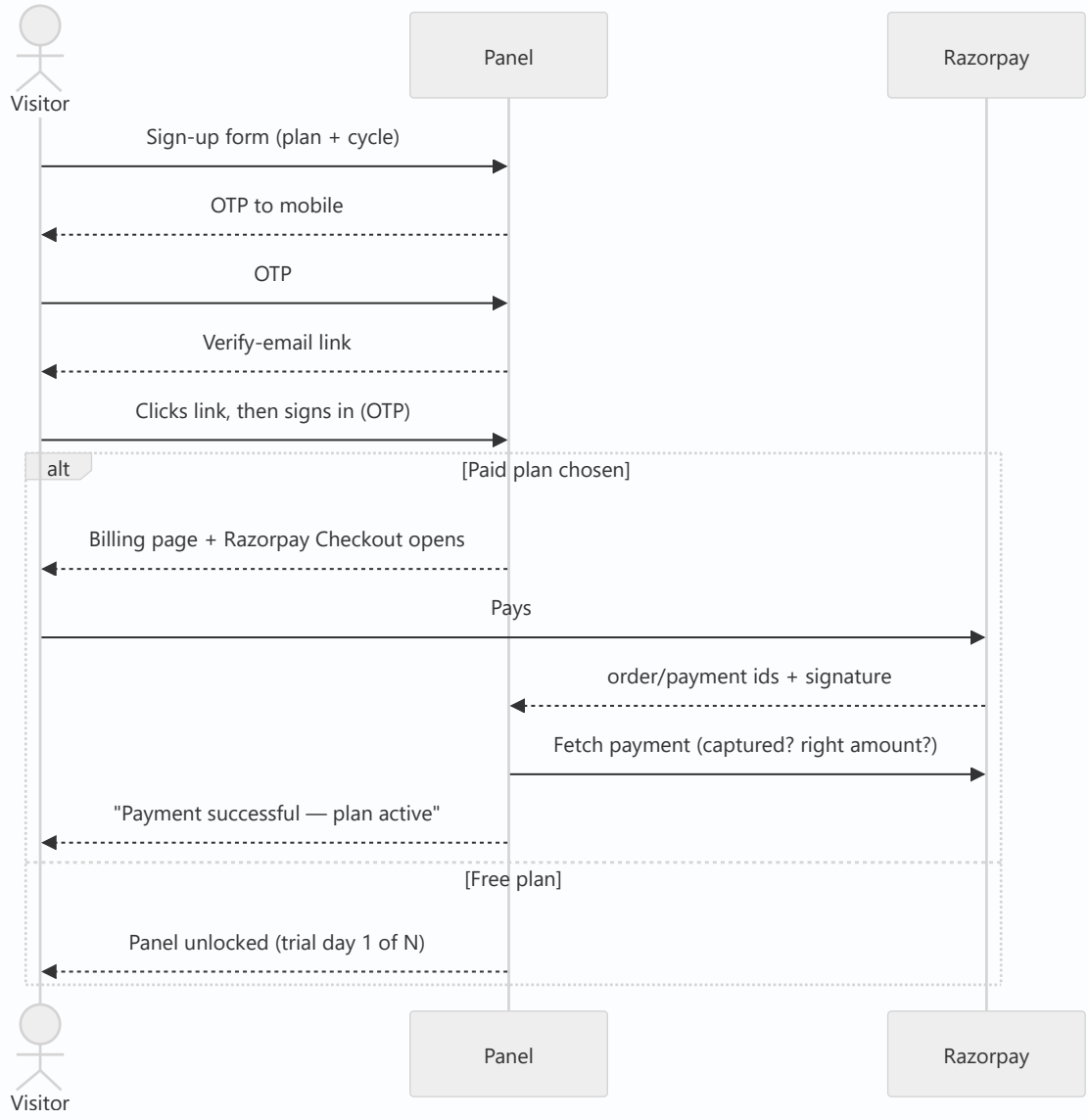
| ID | Requirement |
|---|---|
| FR-P1 | Dashboard: organisations, subscriptions, revenue (this month vs last month), recent payments. |
| FR-P2 | Organisations: create, edit, suspend, delete; set the plan and its expiry; accept or decline requests; record payments; enter an organisation to work inside it. |
| FR-P3 | Plans: create, edit, archive; set prices, currency, limits, features, visibility and the highlight flag. |
| FR-P4 | Admins and users: manage platform admins and move users between organisations. |
| FR-P5 | Leads CRM: list, filter, analytics, notes, status pipeline, locate by IP, convert to an organisation. |

## 2.11 Public site

| ID | Requirement |
|---|---|
| FR-W1 | A server-rendered landing page with live pricing, FAQ, structured data (JSON-LD), sitemap and robots.txt. |
| FR-W2 | Public documentation at `/docs` and `/docs/<slug>`, with the same guides inside the app. |
| FR-W3 | A rate-limited lead form that captures UTM and referrer data and silently ignores bots (honeypot field). |
| FR-W4 | Terms & Conditions page, versioned ( `TERMS_VERSION` ). |

# 3. Key user journeys

## 3.1 Sign up and pay




## 3.2 Bare server to live app

1. **Add server** (SSH credentials), then the panel collects the system profile.
2. **Install Docker** from Installations.
3. **Connect a git account** (OAuth).
4. **+ Custom service**: pick the repo and branch, review the detected settings, then **Build & deploy**.
5. **Add a domain**: Cloudflare DNS + nginx + Let's Encrypt run automatically.
6. Optionally **turn on auto deploy** for the branch.

## 3.3 Run a command on a server

Server page → **Open terminal** → an interactive shell as the server's SSH user. The shell keeps running while the user moves between tabs and closes when they leave the server page.

## 4. Non-functional requirements

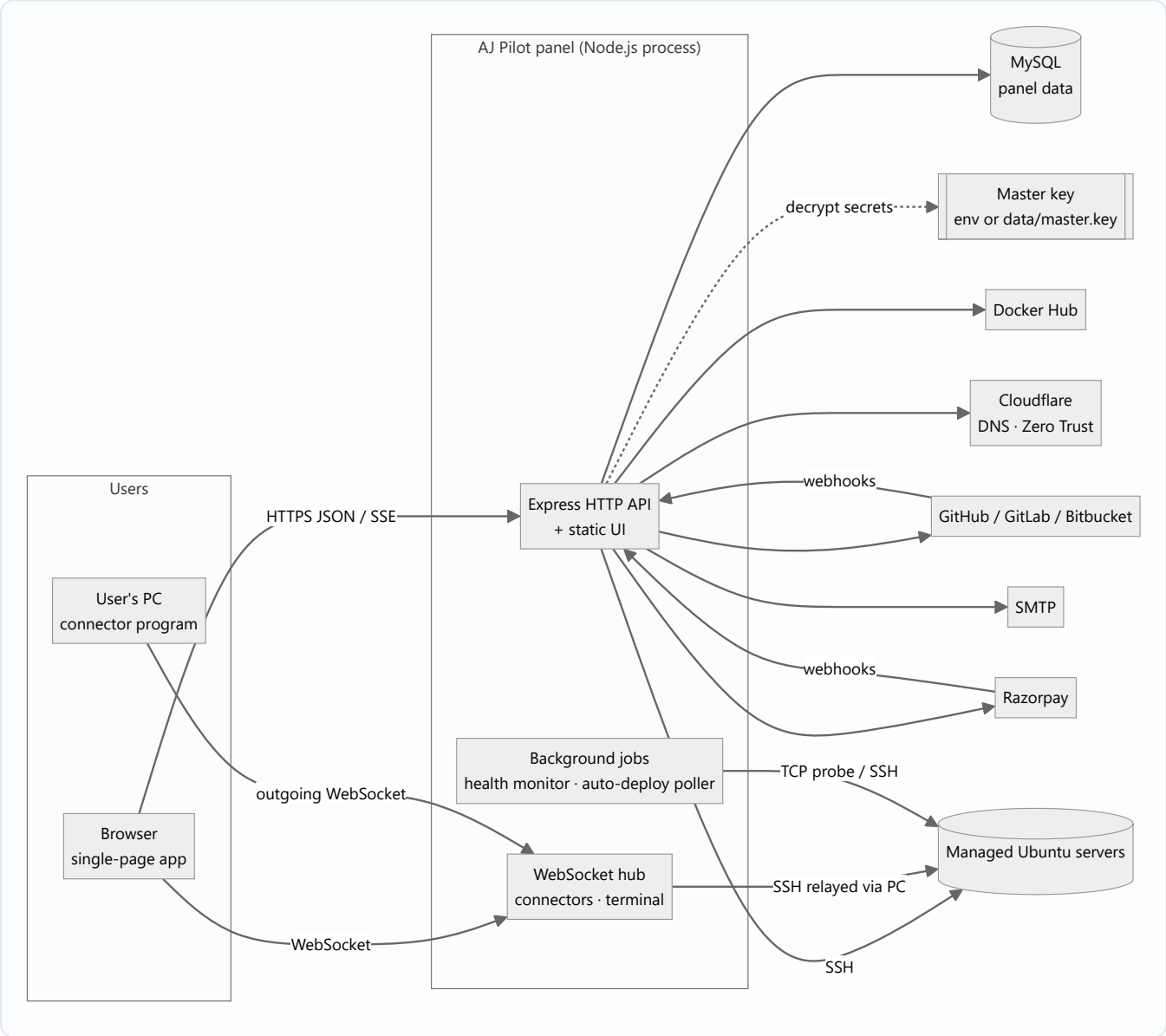
| Area | Requirement |
|---|---|
| Security | Every secret is encrypted with AES-256-GCM at rest; no API returns a stored secret, except where `edit` is explicitly required (environment values, server secrets). |
| Tenancy | Every query is scoped by the requester's organisation. |
| Responsiveness | API calls answer in under a second; long work (deploys, installs) runs in the background with progress shown. |
| Reliability | A failed build leaves the previous release running; payments are idempotent; the health monitor writes only on state changes. |
| Usability | Loading states use skeleton placeholders shaped like the content; the layout works at phone width; light and dark themes. |
| Accessibility | Dialogs trap focus; loading placeholders are announced to screen readers; reduced motion is respected. |
| Portability | No agent on managed servers; only SSH. The UI has no build step. |

CHAPTER 04

# System Design

Architecture, components, request pipeline, key flows

## 1. Context




The panel is the **only** component the product ships. Managed servers need nothing beyond an SSH daemon. Everything that happens on a server is a command or script sent over SSH.

## 2. Technology choices

| Concern | Choice | Reason |
|---|---|---|
| Server runtime | Node.js ≥ 20, Express 4, ES modules | One language end to end; strong SSH and DB client libraries |
| UI | Plain JavaScript SPA in `public/`, no bundler | No build step; served as static files |

| Concern | Choice | Reason |
|---|---|---|
| Panel database | MySQL 8 via `mysql2` pool | Relational data, JSON columns for profiles and settings |
| SSH | `ssh2` | Exec, interactive shells (PTY), custom sockets for the connector relay |
| Real-time | `ws` (WebSocket), SSE | Two-way streams for terminal and connectors; one-way stream for live stats |
| Managed databases | `mysql2`, `pg`, `mongodb`, `redis` | Native clients, optionally over an SSH tunnel |
| Email | `nodemailer` | SMTP; falls back to the server log when unset |
| Terminal UI | `@xterm/xterm` + fit addon | Served from `node_modules` at `/vendor/…`, so the CSP needs no CDN |
| Legacy | `better-sqlite3` | One-time import from the old SQLite store |

## 3. Source layout

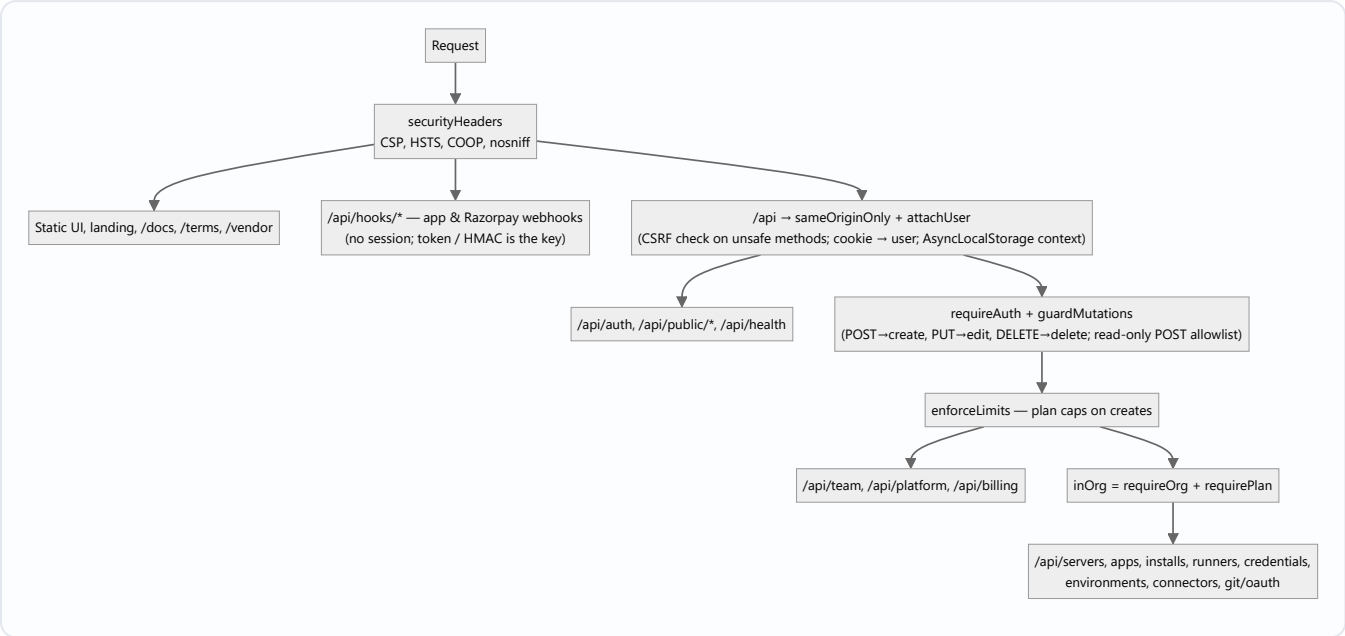
```
src/
  index.js            app wiring: middleware order, routers, webhooks, startup
  config.js           environment → config object, master key
  db/index.js         pool, schema creation, migrations, query helpers, activity log
  db/importSqlite.js  one-time import from the SQLite version
  routes/             HTTP API, one router per area (see API Reference)
  lib/                domain logic, one module per concern (below)
  lib/engines/        PostgreSQL, MongoDB, Redis, Elasticsearch drivers behind one shape
public/
  index.html, app.js, styles.css   the SPA (landing page + panel)
  connector/aj-pilot-connector.mjs the downloadable connector
  docs/                            media for the public guides
docs/                 these documents
```

### 3.1 Library modules by responsibility

| Area | Modules |
|---|---|
| Identity and access | `auth` (scrypt, sessions, roles), `authGuard` (session → user, permissions), `context` (AsyncLocalStorage), `verification` (OTP, email links), `superAdmin`, `terms` |
| Tenancy and commerce | `organisations`, `plans` (subscriptions, limits, trial), `razorpay` (orders, verification, webhook, reconciliation) |
| Server access | `ssh` (connect, route, exec, root exec, host keys), `connectorHub` (PC relay over WebSocket), `netGuard` (blocks private targets), `terminal` (interactive shell) |
| Server knowledge | `probe.sh` + `systemInfo` (profile), `liveStats`, `healthMonitor`, `serverSummary` |
| Server administration | `services` (systemd), `cron`, `sysUsers`, `files`, `nginx`, `nginxLogs`, `docker` |
| Delivery | `nodeApp` (repo inspection, Dockerfile), `deploy` (remote build script), `autoDeploy` (webhooks and polling), `appDomain` (DNS/tunnel + nginx + certificate), `envVars`, `environments`, `catalog` (installable services) |
| Integrations | `git`, `gitAccounts`, `oauth`, `dockerhub`, `cloudflare`, `tunnel`, `runnerInstall`, `mysql`, `engines/*` |
| Public site | `landing`, `docs`, `docsPage`, `geoip`, `device`, `mailer` |
| Cross-cutting | `crypto` (AES-256-GCM), `security` (headers, CSP, same-origin, client IP), `envFile` |

# 4. Request pipeline

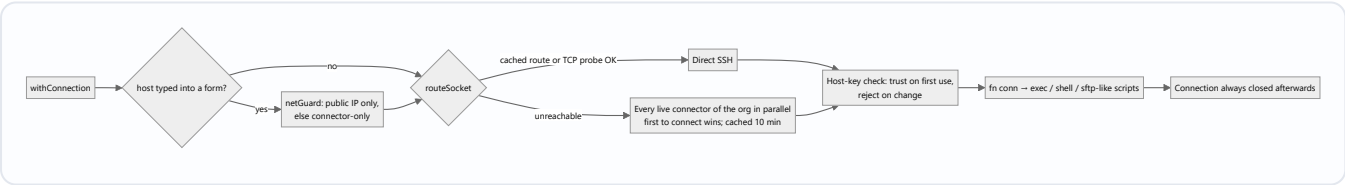
Order matters. It is set in `src/index.js`:




Every organisation-scoped query adds `WHERE org_id = ?` using the requester's organisation. A record from another organisation is reported as *not found*.

# 5. Executing work on a server

## 5.1 Connection routing

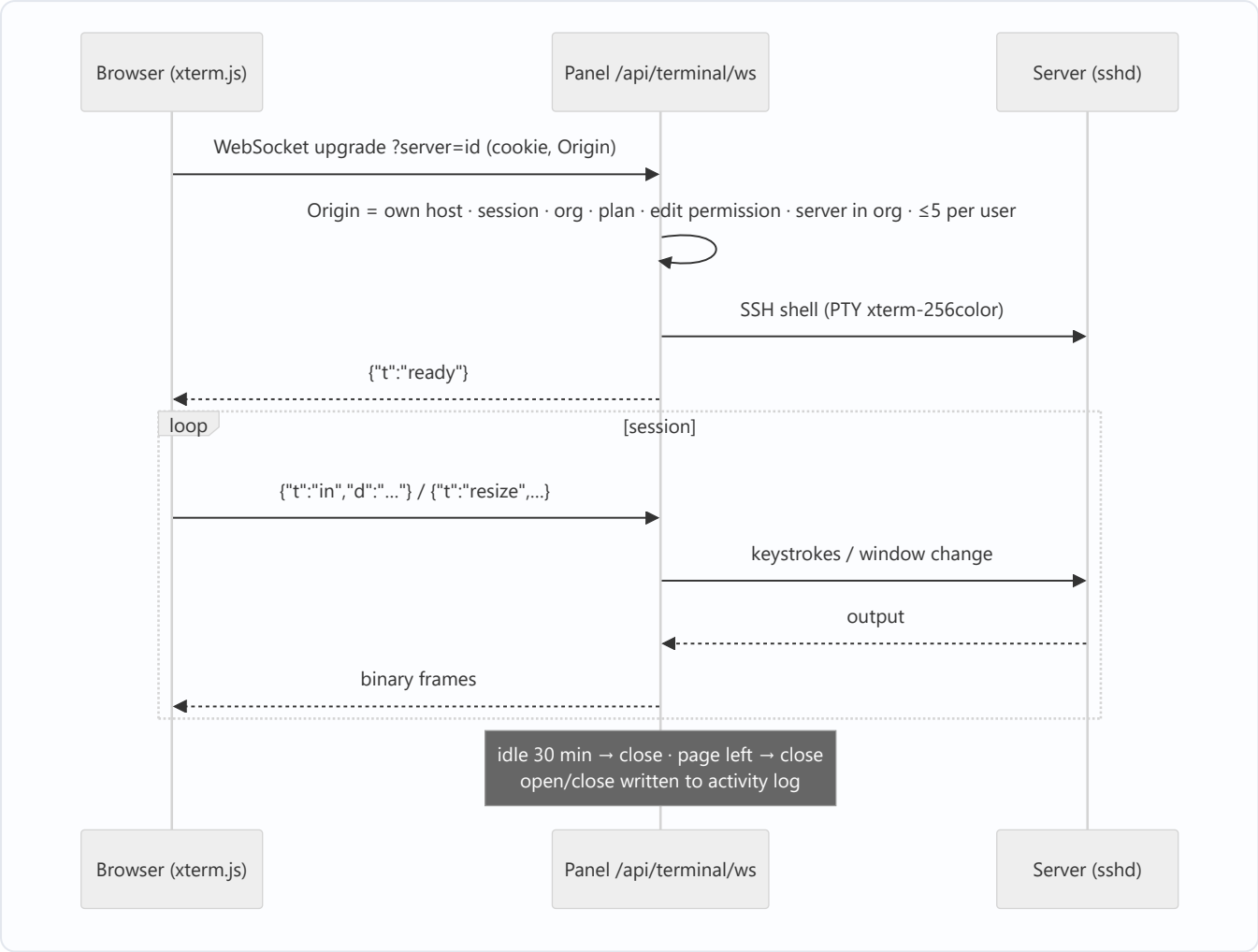



- `exec` runs one command and collects stdout, stderr and the exit code, with a timeout (`SSH_EXEC_TIMEOUT`).
- `rootExec` wraps a script so it runs as root. It uses `sudo` when the login user is not root, with the encrypted sudo password sent on **stdin**. Scripts are passed **base64-encoded**, so form input cannot break out of them.
- Names (units, containers, networks, users) are **validated against strict patterns**, not escaped.

## 5.2 Connectors

A connector is a small Node program on a user's PC. It opens an **outgoing** WebSocket to `/api/connectors/ws`, authenticates with a token (only the token's hash is stored), and then carries multiplexed TCP channels to the hosts the panel asks for. Nothing listens on the PC. One copy runs per token; a new token disconnects the old copy. A heartbeat detects dead links.

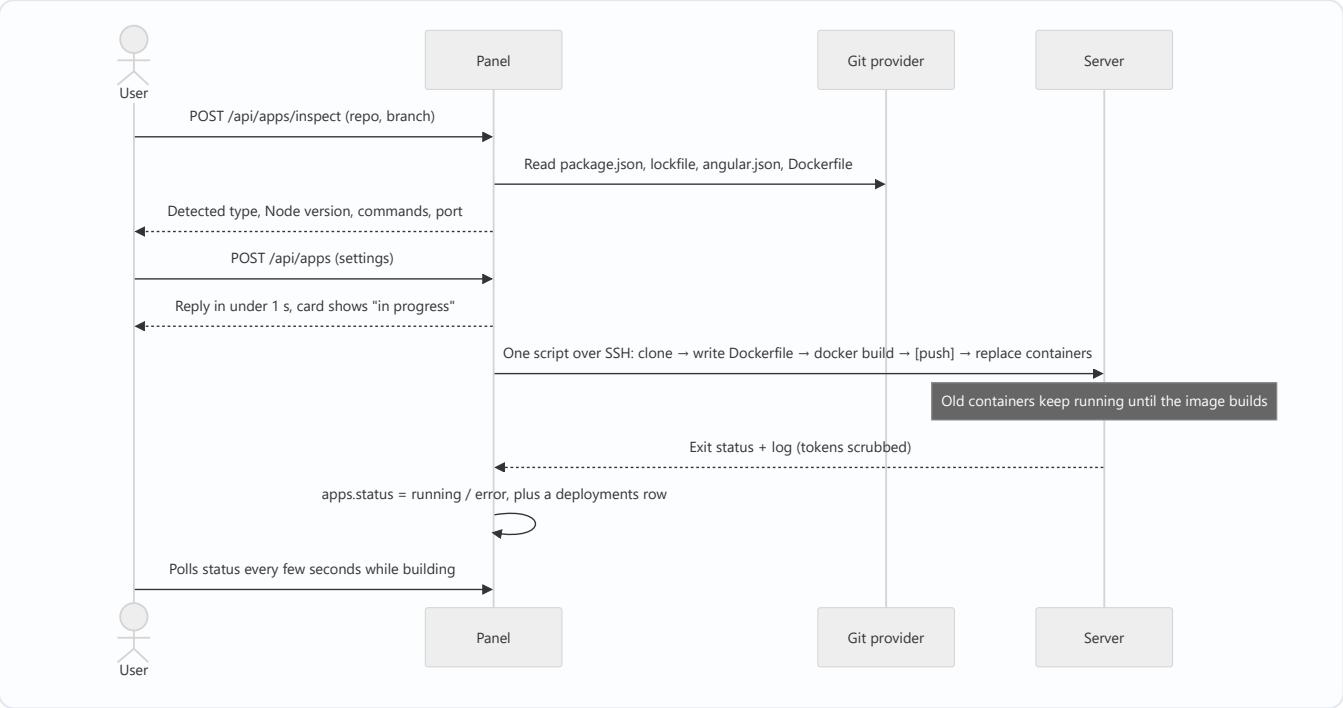
### 5.3 Browser terminal




The connector hub owns the HTTP `upgrade` listener and hands `/api/terminal/ws` to the terminal module.

# 6. Key flows

## 6.1 App deployment




**Auto deploy** reaches the same deploy path from two triggers:

- a **webhook** at `/api/hooks/apps/:token` (GitHub push, GitLab push or merge-request hook, Bitbucket), or
- a **poller** that checks each app's branch head on a timer.

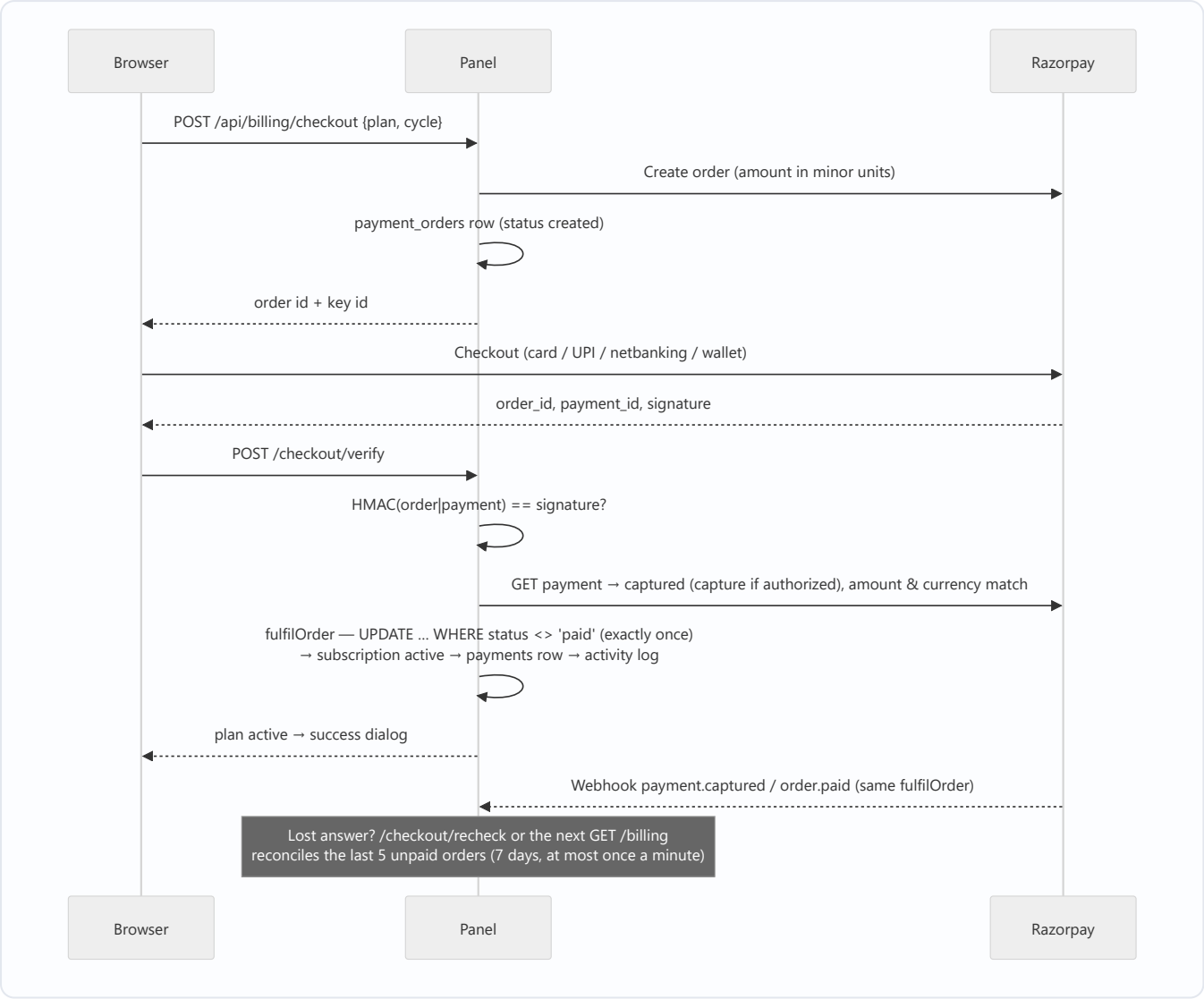
Both check the app's trigger (`push` = every commit, or `merge` = merged pull/merge requests only) and skip a SHA that has already been deployed.

## 6.2 Domain for an app

- **DNS mode:** Cloudflare A record → server's public IP → nginx site proxying to the app's port → Let's Encrypt certificate.
- **Tunnel mode:** Cloudflare Zero Trust tunnel (cloudflared) for servers with no inbound ports.

Each domain is an `app_domains` row with its own status and log, and can be retried or removed.

### 6.3 Plan payment




### 6.4 Health monitoring

- **Tier 1, every** `HEALTH_CHECK_INTERVAL_SECONDS` **(5 s):** a TCP connect to each server's SSH port, `HEALTH_CHECK_CONCURRENCY` at a time. Results are kept in memory. The database and activity log are written **only when the state changes**.
- **Tier 2, every** `HEALTH_CHECK_SIGNIN_MINUTES` **(10 min):** a real SSH sign-in that runs one command, to prove the stored credentials still work.
- The UI polls the lightweight `GET /api/servers/status` every 5 s and patches cards in place.

### 6.5 Live statistics

`GET /api/servers/:id/stats/stream` is a Server-Sent Events stream backed by one SSH session that samples `/proc` twice, one second apart, and computes rates on the machine itself. It is capped at `STATS_MAX_STREAMS` concurrent streams and `STATS_STREAM_MINUTES` per stream; the browser reconnects after that.

**Per app.** `liveStats.collectContainerStats(conn, server, names)` asks only for an app's containers: `docker inspect` (state, start time, restarts, health, OOM-killed) and `docker stats --no-stream` for the running ones, by name. `appUsage()` adds them up and expresses CPU and memory as a share of the host (Docker reports CPU per core, so 200% is two cores).

| Consumer | Endpoint | Cadence |
|---|---|---|
| App cards | `GET /api/apps/stats` — one SSH session per server, every app on it in one pass; a failing server only marks its own apps | Browser polls every 20 s while the list is visible |
| App *Live stats* tab | `GET /api/apps/:id/stats/stream` (SSE), same lifecycle as the server stream, capped by `APP_STATS_STREAMS_PER_ORG` | Every 5 s |

Network and block I/O come back as totals since the container started; the browser turns two samples into a rate.

### 6.6 Statistics history

`lib/metrics.js` runs a recorder every `METRICS_EVERY_MINUTES` (5): for each **online** server, `METRICS_CONCURRENCY` at a time, one `collectLiveStats` pass yields a server row and one row per app on it into `metric_samples`, each with a level (`ok` / `warning` / `critical`, from `THRESHOLDS` and `appLevel`). Rows older than `METRICS_KEEP_DAYS` (35) are pruned hourly. `metricSeries()` buckets them per hour (daily), 6 hours (weekly) or day (monthly) in the viewer's time zone for the Overview *Statistics* charts; `latestLevels()` gives the cards their current level. `METRICS_ENABLED=false` turns the recorder off.

## 7. Multi-tenancy

| Mechanism | Where |
|---|---|
| `org_id` on every owned table; unique names per organisation | `src/db/index.js` (`addOrgScoping` migration) |
| Requester's organisation in AsyncLocalStorage | `authGuard.attachUser` → `context.js` |
| Every route filters by `req.orgId` | all routers (for example `getRow(id, orgId)`) |
| Super admin has no organisation unless one is opened (`active_org_id`) | `orgOf(user)` |
| Plan gate per organisation | `requirePlan`, `enforceLimits`, `blockIfExpired` |
| Connectors are only used for their own organisation's servers | `liveConnectorIds(orgId)` |

## 8. Front-end architecture

- One HTML document holds the landing page, the auth dialogs and every panel view. Views are `<section class="view">` elements that are shown and hidden.
- `app.js` is organised by feature, with section banners. Helpers include `api()` (fetch wrapper with error/plan handling), `show(view)`, `toast`, `askConfirm` / `openAsk` (themed dialogs), `table`, `tile` and `skeleton(kind)` (loading placeholders).
- **Routing:** the hash (`#/servers/12/docker`) mirrors the current page, so refresh, back and bookmarks work.
- **Theming:** CSS custom properties with light and dark sets; the user's choice is kept in `localStorage`.
- **Plan gate:** `body.needs-plan` locks navigation to Billing when there is no active plan.

## 9. Background and in-process state

| State | Lives in | Effect if the process restarts |
|---|---|---|
| Health results | memory map + DB on change | Rebuilt within one sweep |
| Connector sessions | memory map | Connectors reconnect automatically |
| SSH route cache (direct vs connector) | memory map, 10 min TTL | Re-probed on next connection |
| Terminal sessions, live-stat streams | memory | Browser shows disconnected; user reconnects |
| Razorpay reconcile throttle | memory map | Reconciliation may run once more; harmless |

| State | Lives in | Effect if the process restarts |
|---|---|---|
| Running deploys | `apps.deploy_started_at` | Released after 40 min |

Because this state lives in one process, the panel is designed to run as **one instance** (see Limitations).

## 10. Error handling conventions

- JSON errors look like `{ error: "Human-readable sentence" }` with a meaningful status code: 400 validation, 401 not signed in (`needsAuth`), 402 plan required or ended (`needsPlan`, `trialExpired`), 403 permission, 404 not found or not yours, 502 upstream failure (Razorpay, git, Cloudflare).
- SSH errors are mapped to plain-language messages (`normalizeSshError`); a changed host key returns a specific instruction.
- Audit-log writes never fail the request they describe.

CHAPTER 05

# Data Model

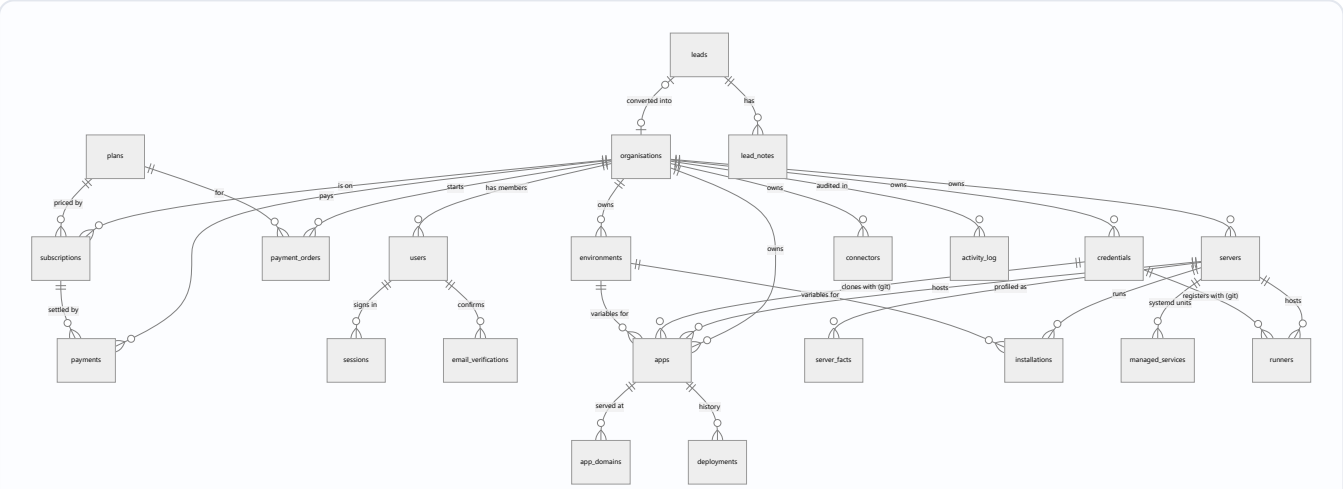
Entities, relationships, tenancy and encryption

The panel keeps all of its data in one MySQL database (`DB_NAME`, default `auto_deploy`). The schema is created and migrated **automatically on boot** by `src/db/index.js`. There is no separate migration tool:

1. `CREATE TABLE IF NOT EXISTS` for each of the 25 tables.
2. `addOrgScoping`: adds `org_id` (+ index) to `servers, credentials, projects, runners, installations, apps, managed_services, activity_log`, and `user_id` to `activity_log`, on databases created before organisations existed. Orphan rows are moved into a "Default organisation".
3. Column additions via `ensureColumn` (for example `servers.host_key`, `users.phone`, `apps.auto_deploy`, `subscriptions.expires_at`).
4. Unique keys added once `org_id` exists: `uq_server_org_name (org_id, name)`, `uq_cred_org_provider_name (org_id, provider, name)`.

> Column types, indexes, constraints and migration rules are in Database Design.
> Columns ending in `_enc` hold **AES-256-GCM ciphertext** (see Security). They are never returned by the API as stored.

## 1. Entity-relationship overview




## 2. Tables by domain

### 2.1 Identity and access

| Table | Purpose | Key columns |
|---|---|---|
| `organisations` | A tenant | `name` (unique), `slug`, `status` (`active` / `suspended`), `notes` |
| `users` | People | `org_id` (null for super admin), `active_org_id` (organisation a super admin has opened), `email`, `password_hash` (scrypt), `role` (`super_admin` / `admin` / `editor` / `viewer`), `status`, `phone`, `email_verified`, `terms_version`, `terms_accepted_at` |

| Table | Purpose | Key columns |
|---|---|---|
| `sessions` | Signed-in sessions | `token_hash` (SHA-256 of the cookie token), `user_id`, `expires_at`, `ip`, `user_agent` |
| `auth_challenges` | OTP challenges for sign-in and sign-up | `token_hash`, `purpose`, `payload` (pending sign-up), `code_hash`, `channel`, `attempts`, `sends`, `expires_at` |
| `email_verifications` | Email confirmation links | `token_hash`, `user_id`, `email`, `expires_at` |

## 2.2 Commerce

| Table | Purpose | Key columns |
|---|---|---|
| `plans` | What is on sale | `name`, `slug`, `price_monthly`, `price_yearly`, `currency`, `limits` (JSON: `servers, apps, users, databases, domains`), `features` (JSON), `highlighted`, `is_public`, `sort_order`, `status` |
| `subscriptions` | An organisation's plan over time | `org_id`, `plan_id`, `cycle`, `amount`, `status` (`active, trial, past_due, pending, declined, withdrawn, cancelled, ended`), `started_at`, `renews_at`, `expires_at`, `ended_at`, `notes` |
| `payments` | Money received (online or recorded by hand) | `org_id`, `subscription_id`, `amount`, `currency`, `paid_at`, `method` (for example `razorpay/upi`, `Bank transfer`), `reference`, `created_by` |
| `payment_orders` | Razorpay orders and their outcome | `org_id`, `plan_id`, `user_id`, `cycle`, `amount`, `provider_order_id` (unique), `provider_payment_id`, `status` (`created` / `paid` / `failed`), `error`, `payment_id` → `payments.id`, `paid_at` |
| `leads` | Prospects from the public form | contact fields, `plan_id`, `cycle`, `utm_*`, `referrer`, `landing_path`, `ip`, `geo`, `device`, `status` (`new → contacted → qualified → proposal → won / lost`), `value`, `assigned_to`, `org_id` once converted |
| `lead_notes` | Notes and events on a lead | `lead_id`, `user_id`, `kind`, `body` |

**The current subscription** is the latest row with status in `active`, `trial` or `past_due`. A `pending` row is a plan request waiting for payment or approval.

## 2.3 Infrastructure

| Table | Purpose | Key columns |
|---|---|---|
| `servers` | Managed machines | `org_id`, `name` (unique per org), `host`, `port`, `username`, `auth_type` (`password` / `key`), `password_enc`, `private_key_enc`, `passphrase_enc`, `sudo_password_enc`, `host_key` (pinned fingerprint), `status`, `last_error`, `last_checked_at` |
| `server_facts` | System profile snapshots | `server_id`, `collected_at`, `duration_ms`, `payload` (JSON) |
| `managed_services` | systemd units the panel created | `server_id`, `unit`, `exec_start`, `unit_file` |
| `installations` | Catalog services on a server | `server_id`, `kind`, `image`, `tag`, `port`, `container_port`, `extra_ports`, `network`, `volume`, `settings`, `secrets_enc`, `environment_id`, `status`, `install_log` |
| `runners` | CI runners | `credential_id` (git), `server_id`, `kind` (GitHub/GitLab), `scope`, `target`, `labels`, `service_name`, `remote_id`, `status`, `install_log` |
| `connectors` | PC relays | `org_id`, `name`, `token_hash`, `hostname`, `platform`, `version`, `last_ip`, `last_seen_at` |

| Table | Purpose | Key columns |
|---|---|---|
| `metric_samples` | CPU, memory, disk, load and network samples for the charts | `org_id`, `kind` (`server` / `app`), `target_id`, `at`, `cpu_pct`, `memory_pct`, `disk_pct`, `load_pct`, `net_rx_bps`, `net_tx_bps`, `level`. Kept for `METRICS_KEEP_DAYS` (35) |

## 2.4 Credentials and configuration

| Table | Purpose | Key columns |
|---|---|---|
| `credentials` | Every external account | `org_id`, `provider` (`git, dockerhub, cloudflare, mysql, postgres, mongodb, redis, elasticsearch`), `name` (unique per org and provider), `username`, `secret_enc`, `extra` (JSON: host, OAuth refresh data, account info), `server_id` (SSH tunnel for databases), `status`, `verified_at` |
| `environments` | Named variable sets | `org_id`, `name` (unique per org), `description`, `env_enc`, `var_count` |
| `projects` | Early placeholder linking a server, credentials, repo and domain | **Unused** by any route; `apps` replaced it. Candidate for removal |

## 2.5 Delivery

| Table | Purpose | Key columns |
|---|---|---|
| `apps` | Deployed services | `org_id`, `server_id`, `credential_id` (git), `registry_cred_id` (Docker Hub), `repo`, `branch`, `root_dir`, `app_type`, `node_version`, `install_cmd` / `build_cmd` / `start_cmd`, `port`, `container_port`, `instances`, `container_ports` (JSON), `volumes` (JSON), `env_enc`, `environment_id`, `image`, `tag`, `pushed`, `network`, `restart`, `use_repo_dockerfile`, `status`, `deploy_started_at`, `deploy_log`, `auto_deploy`, `auto_deploy_trigger` (`push` / `merge`), `webhook_token`, `deployed_sha`, `auto_seen_sha` |
| `deployments` | Each deploy run | `app_id`, `reason`, `user_id`, `branch`, `kind` (`commit` / `merge`), `sha`, `message`, `author`, `pr_number`, `pr_title`, `status`, `error`, `log`, `started_at`, `finished_at` |
| `app_domains` | Domains attached to apps | `org_id`, `app_id`, `domain` (unique per org), `port`, `config` (mode: dns / tunnel), `status`, `error`, `log` |

## 2.6 Monitoring

| Table | Purpose | Key columns |
|---|---|---|
| `metric_samples` | One sample per server and per app every `METRICS_EVERY_MINUTES`, for the Statistics charts and the cards' health level; pruned after `METRICS_KEEP_DAYS` | `org_id`, `kind` (`server` / `app`), `target_id`, `at` (UTC), `cpu_pct`, `memory_pct`, `disk_pct` (servers), `load_pct` (servers, load per core × 100), `net_rx_bps` / `net_tx_bps` (servers), `level` (`ok` / `warning` / `critical`) |

Live statistics (server *Live*, app *Live stats*, app card usage) are read over SSH on demand and never stored.

## 2.7 Audit

| Table | Purpose | Key columns |
|---|---|---|
| `activity_log` | Every change and notable event | `entity`, `entity_id`, `action` (for example `plan_chosen`, `payment_online`, `went_offline`, `terminal_opened`), `level` (`info` / `warn` / `error`), `message`, `org_id`, `user_id`, `created_at` |

`org_id` and `user_id` are filled automatically from the request context, so callers never pass them.

## 3. Integrity and deletion

- Foreign keys cascade from `organisations` (users, subscriptions, payments, payment orders, environments). Deleting an organisation removes its people, subscriptions and payment records.
- Server-owned rows (`server_facts`, `installations`, `runners`, `managed_services`) reference `servers`. Deleting a server in the panel does **not** remove anything from the machine itself.
- Payment idempotency: `UPDATE payment_orders SET status='paid' … WHERE id = ? AND status <> 'paid'`. Only the call that changes the row goes on to activate the plan.

## 4. Data that is *not* in the database

| Data | Where it lives |
|---|---|
| Master encryption key | `APP_ENCRYPTION_KEY`, or `data/master.key` |
| Nginx sites, certificates, crontabs, systemd units, Docker state | On each managed server; the panel reads them live and keeps no copy |
| App source and images | On the managed server (`/opt/auto-deploy/apps/<name>`), and Docker Hub if pushed |
| Logs | Read live over SSH; build logs are stored in `apps.deploy_log` / `deployments.log` with tokens scrubbed |

## 5. Backup guidance

- Back up the **MySQL database** and the **master key**, separately. Either one alone is useless for restoring credentials.
- A `mysqldump --single-transaction` of `auto_deploy` is consistent with no downtime.
- After a restore with the same key, every credential works again. With a different key, every `_enc` column is unreadable.

CHAPTER 06

# Database Design

Every table, column, key and index in detail

This document is the physical design of the panel's MySQL database: every table, column, type, key and constraint, and the rules the code follows when it reads and writes them. Data Model is the conceptual overview. Read that first, then use this document when you write a query, add a column or review a migration.

The source of truth is `createSchema()` and `addOrgScoping()` in `src/db/index.js`. When this document and that file disagree, the file is right.

## 1. Engine and conventions

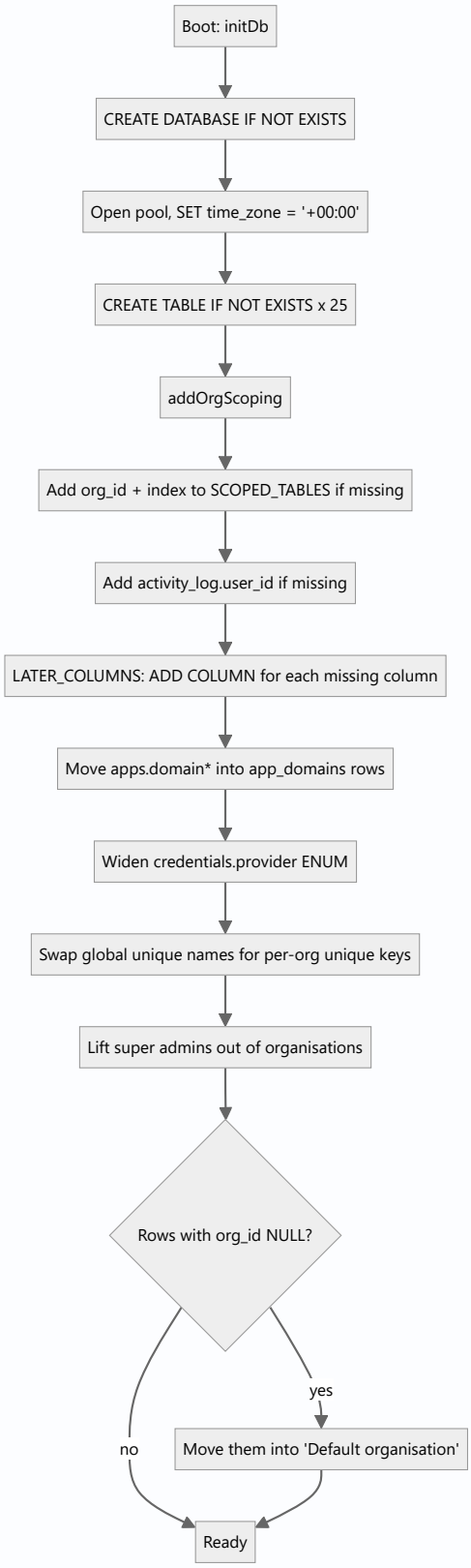
| Setting | Value | Why |
|---|---|---|
| Server | MySQL 8 | `JSON` columns, `CHECK`-free ENUMs, `information_schema` probing for migrations |
| Database | `DB_NAME` (default `auto_deploy`), created on boot if missing | No manual setup step |
| Engine | InnoDB on every table | Transactions, row locks, foreign keys |
| Charset / collation | `utf8mb4` / `utf8mb4_unicode_ci` | Full Unicode (emoji in commit messages, names). Comparisons ignore case, so `UNIQUE` on `email` is case-insensitive |
| Time zone | Session `time_zone = '+00:00'`, pool `timezone: 'Z'`, `dateStrings: true` | Every `DATETIME` is UTC. Dates come back as strings, and the browser converts them to local time |
| Primary keys | `id INT AUTO_INCREMENT` (`BIGINT` for `metric_samples`) | Simple surrogate keys. Only `metric_samples` is expected to pass 2 billion rows |
| Timestamps | `created_at DEFAULT CURRENT_TIMESTAMP`, `updated_at … ON UPDATE CURRENT_TIMESTAMP` | Maintained by MySQL, not by the code |
| Booleans | `TINYINT(1)` with `0/1` | |
| Status fields | Mostly `VARCHAR(20–30)`, not `ENUM` | New states can be added without an `ALTER`. Only `users.role`, `users.status`, `servers.auth_type`, `credentials.provider`, `runners.kind/scope` and `metric_samples.kind` are real ENUMs |
| Secrets | Columns ending `_enc` hold AES-256-GCM ciphertext. Tokens are stored as SHA-256 (`token_hash`, `code_hash`) | A database dump alone reveals no secret and no live token |
| Large text | `MEDIUMTEXT` (up to 16 MB) for logs and unit files, `TEXT` (64 KB) for errors and ciphertext | |

### Naming

- Tables are plural `snake_case`. Columns are `snake_case`.
- Foreign-key columns are `<entity>_id`. Constraints are `fk_<table>_<target>`, unique keys `uq_<…>`, plain indexes `idx_<…>`.
- Tenant column is always `org_id`.

## 2. Schema lifecycle

There is no migration tool. Every boot runs the same idempotent sequence, so a fresh database and a two-year-old one end up identical.



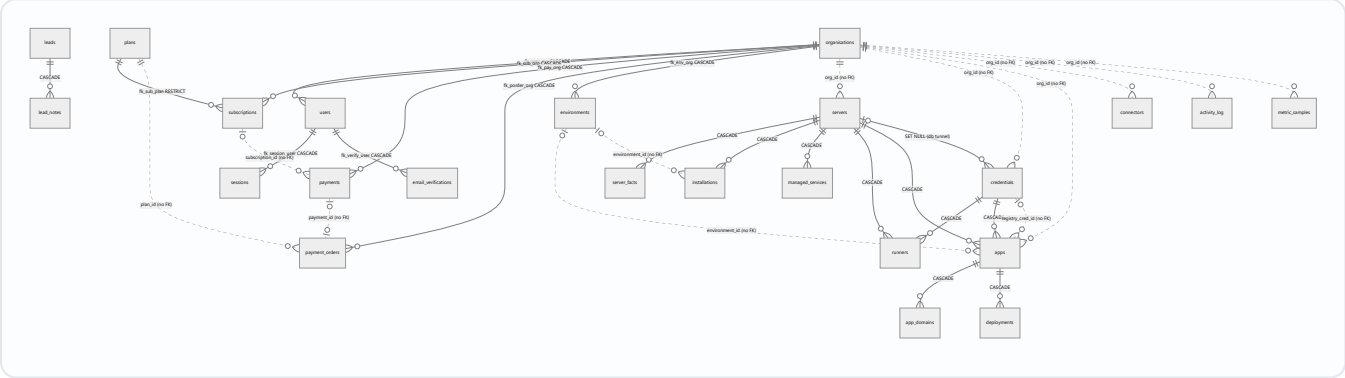

### How to change the schema

| Change | Do this |
|---|---|
| New table | Add a `CREATE TABLE IF NOT EXISTS` to `createSchema()`. If its rows belong to a tenant, include `org_id` and an index in the definition |
| New column on an existing table | Add it to the `CREATE TABLE` **and** append `[table, column, definition]` to `LATER_COLUMNS`. The first covers new installs, the second existing ones |
| New credential provider | Add it to `PROVIDERS`. The ENUM is widened on next boot |
| New index or unique key | Probe `information_schema.statistics` first, then `ALTER TABLE … ADD`. Copy the `uq_server_org_name` block |
| Rename / drop a column | Not supported automatically. Write a guarded one-off step in `addOrgScoping()` and keep old reads working for one release |

Every step must be safe to run twice and safe against a database that is several releases behind.

## 3. Entity-relationship diagram (physical)

Solid lines are enforced foreign keys. Relationships drawn with `..` exist in the code only (see §6).



## 4. Table definitions

Legend: **PK** primary key, **FK** foreign key, **UQ** unique, **IX** index, **N** nullable. Columns marked *(later)* are added by `LATER_COLUMNS` on older databases.

### 4.1 Identity and access

`organisations` **— a tenant**

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| name | VARCHAR(190) | | | UQ |
| slug | VARCHAR(190) | | | UQ |
| notes | TEXT | ✓ | | |
| status | VARCHAR(20) | | `'active'` | *(later)* `active` / `suspended`. Suspended members cannot sign in |
| created_at, updated_at | DATETIME | | now | |

`users`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| org_id | INT | ✓ | | FK → organisations ON DELETE CASCADE. IX `idx_user_org`. **NULL for super admins** |
| active_org_id | INT | ✓ | | The organisation a super admin has opened. No FK |
| email | VARCHAR(190) | | | UQ (case-insensitive by collation) |
| name | VARCHAR(190) | | | |
| password_hash | TEXT | | | scrypt |
| role | ENUM | | `'viewer'` | `super_admin`, `admin`, `editor`, `viewer` |
| status | ENUM | | `'active'` | `active`, `disabled` |
| phone | VARCHAR(32) | ✓ | | *(later)* Required for admins and super admins (enforced in code) |
| email_verified | TINYINT(1) | | 1 | *(later)* Self sign-ups start at 0 |
| email_verified_at | DATETIME | ✓ | | *(later)* |
| terms_version | VARCHAR(20) | ✓ | | *(later)* |
| terms_accepted_at | DATETIME | ✓ | | *(later)* |
| last_login_at | DATETIME | ✓ | | |
| created_by | INT | ✓ | | users.id, no FK |
| created_at, updated_at | DATETIME | | now | |

`sessions`

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| token_hash | CHAR(64) | | UQ. SHA-256 of the cookie token. The raw token is never stored |
| user_id | INT | | FK → users CASCADE. IX `idx_session_user` |
| expires_at | DATETIME | | Checked on every request |
| ip | VARCHAR(64) | ✓ | |
| user_agent | VARCHAR(255) | ✓ | |
| created_at | DATETIME | | |

### `auth_challenges` — pending one-time codes

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| token_hash | CHAR(64) | | | UQ. Identifies the challenge to the browser |
| purpose | VARCHAR(16) | | | Sign-in or sign-up |
| user_id | INT | ✓ | | Set for sign-in. No FK |
| payload | MEDIUMTEXT | ✓ | | Encrypted pending sign-up (the user row does not exist yet) |
| code_hash | CHAR(64) | | | SHA-256 of the OTP |
| channel | VARCHAR(10) | | | `email` / `sms` |
| sent_to | VARCHAR(190) | | | |
| attempts | INT | | 0 | Wrong guesses |
| sends | INT | | 1 | Resends |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| last_sent_at | DATETIME | | now | Resend throttle |
| expires_at | DATETIME | | | IX `idx_challenge_expiry`. Expired rows are deleted on each new challenge |
| created_at | DATETIME | | now | |

`email_verifications`

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| token_hash | CHAR(64) | | UQ |
| user_id | INT | | FK → users CASCADE. IX `idx_verify_user` |
| email | VARCHAR(190) | | The address being confirmed |
| expires_at | DATETIME | | |
| created_at | DATETIME | | |

## 4.2 Commerce

`plans`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| name | VARCHAR(120) | | | |
| slug | VARCHAR(120) | | | UQ |
| tagline | VARCHAR(255) | ✓ | | |
| price_monthly, price_yearly | DECIMAL(12,2) | | 0 | Money is never stored as a float |
| currency | VARCHAR(8) | | `'INR'` | |
| limits | JSON | ✓ | | See §5 |
| features | JSON | ✓ | | Array of strings shown on the pricing page |
| highlighted | TINYINT(1) | | 0 | |
| is_public | TINYINT(1) | | 1 | Hidden plans can still be assigned by a super admin |
| sort_order | INT | | 0 | |
| status | VARCHAR(20) | | `'active'` | |
| created_at, updated_at | DATETIME | | now | |

`subscriptions`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | | | FK → organisations CASCADE. IX `idx_sub_org` |
| plan_id | INT | | | FK → plans (RESTRICT: a plan in use cannot be deleted). IX `idx_sub_plan` |
| cycle | VARCHAR(10) | | `'monthly'` | `monthly` / `yearly` |
| amount | DECIMAL(12,2) | | 0 | Price at the time of purchase, so later plan price changes do not rewrite history |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| currency | VARCHAR(8) | | `'INR'` | |
| status | VARCHAR(20) | | `'active'` | `active, trial, past_due, pending, declined, withdrawn, cancelled, ended` |
| started_at | DATETIME | | now | |
| renews_at | DATETIME | ✓ | | |
| expires_at | DATETIME | ✓ | | *(later)* After this the organisation is view-only |
| ended_at | DATETIME | ✓ | | |
| notes | TEXT | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

### `payments` — money received

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| org_id | INT | | FK → organisations CASCADE. IX `idx_pay_org` |
| subscription_id | INT | ✓ | subscriptions.id, no FK |
| amount | DECIMAL(12,2) | | |
| currency | VARCHAR(8) | | default `'INR'` |
| paid_at | DATETIME | | IX `idx_pay_paid` (revenue reports) |
| method | VARCHAR(40) | ✓ | e.g. `razorpay/upi`, `Bank transfer` |
| reference | VARCHAR(190) | ✓ | Gateway payment id or bank reference |
| notes | TEXT | ✓ | |
| created_by | INT | ✓ | users.id of whoever recorded it; NULL for online payments |
| created_at | DATETIME | | |

### `payment_orders` — Razorpay orders

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | | | FK → organisations CASCADE. IX `idx_porder_org` |
| plan_id | INT | | | plans.id, no FK |
| user_id | INT | ✓ | | Who started checkout |
| cycle | VARCHAR(10) | | `'monthly'` | |
| amount | DECIMAL(12,2) | | | |
| currency | VARCHAR(8) | | `'INR'` | |
| provider | VARCHAR(20) | | `'razorpay'` | |
| provider_order_id | VARCHAR(64) | | | UQ. Look-up key for webhooks and the checkout callback |
| provider_payment_id | VARCHAR(64) | ✓ | | |
| status | VARCHAR(20) | | `'created'` | `created → paid` or `created → failed` |
| error | VARCHAR(500) | ✓ | | |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| payment_id | INT | ✓ | | payments.id once paid, no FK |
| paid_at | DATETIME | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

`leads`

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| name, email | VARCHAR(190) | | IX `idx_lead_email` |
| phone | VARCHAR(40) | ✓ | |
| company | VARCHAR(190) | ✓ | |
| servers | VARCHAR(20) | ✓ | Self-reported fleet size band |
| plan_id | INT | ✓ | No FK |
| cycle | VARCHAR(10) | ✓ | |
| message | TEXT | ✓ | |
| form | VARCHAR(30) | | default `'contact'` |
| source | VARCHAR(60) | ✓ | |
| utm_source / medium / campaign / term / content | VARCHAR(120) | ✓ | |
| referrer, landing_path | VARCHAR(500) | ✓ | |
| ip | VARCHAR(64) | ✓ | |
| user_agent | VARCHAR(255) | ✓ | |
| device, visit, geo | JSON | ✓ | *(later)* |
| visitor_id | VARCHAR(64) | ✓ | *(later)* |
| fingerprint | CHAR(64) | ✓ | *(later)* |
| country | VARCHAR(2) | ✓ | *(later)* ISO code |
| status | VARCHAR(20) | | `new → contacted → qualified → proposal → won / lost`. IX `idx_lead_status` |
| value | DECIMAL(12,2) | ✓ | Expected deal size |
| assigned_to | INT | ✓ | users.id, no FK |
| lost_reason | VARCHAR(190) | ✓ | |
| org_id | INT | ✓ | Set when converted into an organisation |
| contacted_at, closed_at | DATETIME | ✓ | |
| created_at, updated_at | DATETIME | | IX `idx_lead_created` |

`lead_notes`

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| lead_id | INT | | FK → leads CASCADE. IX `idx_note_lead` |
| user_id | INT | ✓ | Author |
| kind | VARCHAR(20) | | default `'note'`; also status-change events |

| Column | Type | N | Notes |
|---|---|---|---|
| body | TEXT | | |
| created_at | DATETIME | | |

### 4.3 Infrastructure

`servers`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | ✓ | | *(later)* IX `idx_servers_org`. UQ `uq_server_org_name (org_id, name)` |
| name | VARCHAR(190) | | | |
| host | VARCHAR(255) | | | IP or hostname |
| port | INT | | 22 | |
| username | VARCHAR(190) | | | |
| auth_type | ENUM | | `'password'` | `password`, `key` |
| password_enc, private_key_enc, passphrase_enc, sudo_password_enc | TEXT | ✓ | | Encrypted |
| host_key | VARCHAR(128) | ✓ | | *(later)* Fingerprint pinned on first connect |
| tags | VARCHAR(500) | ✓ | | Comma-separated |
| notes | TEXT | ✓ | | |
| status | VARCHAR(30) | | `'unknown'` | `online / offline / unknown` |
| last_error | TEXT | ✓ | | |
| last_checked_at | DATETIME | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

> The `CREATE TABLE` still declares `name … UNIQUE`. On boot that single-column unique index is dropped and replaced by `uq_server_org_name`.

#### `server_facts` — profile snapshots

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| server_id | INT | | FK → servers CASCADE |
| collected_at | DATETIME | | IX `idx_server_collected (server_id, collected_at DESC)` |
| duration_ms | INT | ✓ | |
| payload | JSON | | OS, CPU, memory, disks, installed software |

Latest snapshot per server is read with `WHERE id IN (SELECT MAX(id) … GROUP BY server_id)`.

#### `installations` — catalog services (databases, caches, tools) on a server

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| org_id | INT | ✓ | | *(later)* IX |
| server_id | INT | | | FK → servers CASCADE. UQ `uq_install_name (server_id, name)` |
| kind | VARCHAR(40) | | | Catalog key, e.g. `mysql`, `redis` |
| name | VARCHAR(190) | | | Container name |
| image, tag | VARCHAR | ✓ | | |
| port, container_port | INT | ✓ | | Host and container ports |
| extra_ports | JSON | ✓ | | *(later)* |
| network, volume | VARCHAR(190) | ✓ | | Docker network and named volume |
| container_id | VARCHAR(90) | ✓ | | |
| settings | JSON | ✓ | | Non-secret options |
| secrets_enc | TEXT | ✓ | | Encrypted passwords |
| environment_id | INT | ✓ | | *(later)* No FK |
| status | VARCHAR(30) | | `'pending'` | |
| last_error | TEXT | ✓ | | |
| install_log | MEDIUMTEXT | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

### `managed_services` — systemd units created by the panel

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| org_id | INT | ✓ | *(later)* IX |
| server_id | INT | | FK → servers CASCADE. UQ `uq_service_unit (server_id, unit)` |
| unit | VARCHAR(190) | | e.g. `myworker.service` |
| description | VARCHAR(255) | ✓ | |
| exec_start | TEXT | ✓ | |
| unit_file | MEDIUMTEXT | ✓ | Full text as written to the server |
| created_at, updated_at | DATETIME | | |

### `runners` — GitHub / GitLab CI runners

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | ✓ | | *(later)* IX |
| name | VARCHAR(190) | | | UQ `uq_runner_target_name (credential_id, target, name)` |
| credential_id | INT | | | FK → credentials CASCADE (a git credential) |
| server_id | INT | | | FK → servers CASCADE. IX `idx_runner_server` |
| kind | ENUM | | `'github'` | `github`, `gitlab` |
| scope | ENUM | | `'repo'` | `repo`, `org` |
| target | VARCHAR(255) | | | `owner/repo` or org name |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| labels | VARCHAR(500) | ✓ | | |
| runner_dir, service_name, service_user | VARCHAR | ✓ | | Where and how it runs on the server |
| remote_id | VARCHAR(60) | ✓ | | The runner's id at GitHub/GitLab |
| status | VARCHAR(30) | | `'pending'` | |
| last_error | TEXT | ✓ | | |
| install_log | MEDIUMTEXT | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

### `connectors` — PC relays

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| org_id | INT | ✓ | IX `idx_connectors_org` |
| name | VARCHAR(190) | | |
| token_hash | CHAR(64) | | UQ `uq_connector_token` |
| hostname, platform, version | VARCHAR | ✓ | Reported by the connector on connect |
| last_ip | VARCHAR(64) | ✓ | |
| last_seen_at | DATETIME | ✓ | |
| created_at | DATETIME | | |

### `metric_samples` — time series for charts

| Column | Type | N | Notes |
|---|---|---|---|
| id | BIGINT | | PK |
| org_id | INT | ✓ | |
| kind | ENUM | | `server`, `app` |
| target_id | INT | | servers.id or apps.id, depending on `kind`. No FK (polymorphic) |
| at | DATETIME | | Sample time (UTC) |
| cpu_pct, memory_pct, disk_pct | DECIMAL(6,1) | ✓ | 0–100 |
| load_pct | DECIMAL(7,1) | ✓ | Load ÷ cores × 100, can exceed 100 |
| net_rx_bps, net_tx_bps | BIGINT | ✓ | |
| level | VARCHAR(10) | | default `'ok'` |

Indexes: `idx_metric_target (kind, target_id, at)` serves every chart query; `idx_metric_at (at)` serves the retention delete.

## 4.4 Credentials and configuration

`credentials`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| org_id | INT | ✓ | | *(later)* IX. UQ `uq_cred_org_provider_name (org_id, provider, name)` |
| provider | ENUM | | | `git, dockerhub, cloudflare, mysql, postgres, mongodb, redis, elasticsearch` (from `PROVIDERS`) |
| name | VARCHAR(190) | | | |
| username | VARCHAR(190) | ✓ | | |
| secret_enc | TEXT | | | Encrypted token, password or OAuth access token |
| extra | JSON | ✓ | | See §5 |
| server_id | INT | ✓ | | FK → servers ON DELETE SET NULL. SSH tunnel for database credentials |
| status | VARCHAR(30) | | `'unverified'` | `unverified / verified / error` |
| last_error | TEXT | ✓ | | |
| verified_at | DATETIME | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

### `environments` — named variable sets

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| org_id | INT | | FK → organisations CASCADE. UQ `uq_env_name (org_id, name)` |
| name | VARCHAR(120) | | |
| description | VARCHAR(255) | ✓ | |
| env_enc | MEDIUMTEXT | ✓ | Encrypted `KEY=value` set |
| var_count | INT | | Denormalised so lists need no decryption |
| created_by | INT | ✓ | |
| created_at, updated_at | DATETIME | | |

### `projects` — legacy, unused

`id, org_id, name (UQ), server_id, git_cred_id, docker_cred_id, cf_cred_id, repo_url, branch, domain, config JSON, created_at, updated_at`, with `SET NULL` foreign keys to `servers` and `credentials`. No route reads or writes it. `apps` replaced it. It is a candidate for removal (see Limitations & Roadmap).

## 4.5 Delivery

`apps`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | ✓ | | IX `idx_app_org` |
| server_id | INT | | | FK → servers CASCADE. UQ `uq_app_name (server_id, name)` |
| credential_id | INT | | | FK → credentials CASCADE (git) |
| registry_cred_id | INT | ✓ | | Docker Hub credential. No FK |
| environment_id | INT | ✓ | | *(later)* No FK |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| name | VARCHAR(190) | | | |
| repo | VARCHAR(255) | | | |
| branch | VARCHAR(190) | | | |
| root_dir | VARCHAR(255) | ✓ | | *(later)* Monorepo subfolder |
| app_type | VARCHAR(30) | | `'node'` | *(later)* Build strategy |
| node_version | VARCHAR(20) | ✓ | | |
| install_cmd, build_cmd, start_cmd | VARCHAR(500) | ✓ | | |
| output_dir | VARCHAR(190) | ✓ | | *(later)* Static builds |
| port | INT | | | Host port |
| container_port | INT | | | |
| instances | INT | | 1 | *(later)* Replica count |
| container_ports | JSON | ✓ | | *(later)* Per-instance host ports |
| containers | JSON | ✓ | | *(later)* Running container ids |
| volumes | JSON | ✓ | | *(later)* |
| image, tag | VARCHAR | ✓ | | Built image |
| image_bytes | BIGINT | ✓ | | |
| pushed | TINYINT(1) | | 0 | Pushed to Docker Hub |
| network | VARCHAR(190) | ✓ | | |
| restart | VARCHAR(30) | | `'unless-stopped'` | Docker restart policy |
| use_repo_dockerfile | TINYINT(1) | | 0 | |
| env_enc | TEXT | ✓ | | Encrypted app-specific variables |
| detected | JSON | ✓ | | What repo inspection found (framework, package manager…) |
| container_id | VARCHAR(90) | ✓ | | |
| status | VARCHAR(30) | | `'pending'` | `pending, deploying, running, configuring, error, failed, interrupted, removing, remove_failed` |
| last_error | TEXT | ✓ | | |
| deploy_log | MEDIUMTEXT | ✓ | | Latest deploy, tokens scrubbed |
| deploy_started_at | DATETIME | ✓ | | *(later)* Detects deploys interrupted by a restart |
| last_deployed_at | DATETIME | ✓ | | |
| auto_deploy | TINYINT(1) | | 0 | *(later)* |
| auto_deploy_trigger | VARCHAR(10) | | `'push'` | *(later)* `push` / `merge` |
| webhook_token | VARCHAR(64) | ✓ | | *(later)* Secret in the webhook URL |
| webhook_id | VARCHAR(64) | ✓ | | *(later)* Id at the git host |
| deployed_sha, deployed_message | VARCHAR | ✓ | | *(later)* |
| auto_seen_sha | VARCHAR(64) | ✓ | | *(later)* Last commit the poller saw |
| auto_checked_at | DATETIME | ✓ | | *(later)* |
| auto_error | TEXT | ✓ | | *(later)* |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| domain, domain_config, domain_status, domain_error, domain_log | | ✓ | | *(later)* **Deprecated.** Moved into `app_domains` on boot and set to NULL |
| created_at, updated_at | DATETIME | | now | |

`app_domains`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | ✓ | | UQ `uq_app_domain (org_id, domain)`: one org cannot attach a domain twice |
| app_id | INT | | | FK → apps CASCADE. IX `idx_domain_app` |
| domain | VARCHAR(255) | | | |
| port | INT | ✓ | | Which app port it serves, for multi-port apps |
| config | JSON | ✓ | | See §5 |
| status | VARCHAR(30) | | `'pending'` | |
| error | TEXT | ✓ | | |
| log | MEDIUMTEXT | ✓ | | |
| created_at, updated_at | DATETIME | | now | |

`deployments`

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| id | INT | | auto | PK |
| org_id | INT | ✓ | | |
| app_id | INT | | | FK → apps CASCADE. IX `idx_deploy_app (app_id, id)` |
| trigger | VARCHAR(20) | | `'manual'` | Reserved word, always quoted with backticks. `first` (initial deploy), `manual`, or the auto-deploy trigger passed in `meta.trigger` |
| reason | VARCHAR(500) | ✓ | | |
| user_id | INT | ✓ | | |
| branch | VARCHAR(190) | ✓ | | |
| kind | VARCHAR(10) | ✓ | | `commit` / `merge` |
| sha | VARCHAR(64) | ✓ | | |
| message | VARCHAR(255) | ✓ | | Truncated to 255 in code |
| author | VARCHAR(190) | ✓ | | |
| committed_at | DATETIME | ✓ | | |
| pr_number | VARCHAR(20) | ✓ | | |
| pr_title | VARCHAR(255) | ✓ | | |
| pr_from | VARCHAR(190) | ✓ | | Source branch |
| status | VARCHAR(20) | | `'running'` | |
| error | TEXT | ✓ | | |
| log | MEDIUMTEXT | ✓ | | |

| Column | Type | N | Default | Notes |
|---|---|---|---|---|
| started_at | DATETIME | | now | |
| finished_at | DATETIME | ✓ | | |

### 4.6 Audit

`activity_log`

| Column | Type | N | Notes |
|---|---|---|---|
| id | INT | | PK |
| org_id | INT | ✓ | *(later)* IX `idx_activity_log_org` |
| user_id | INT | ✓ | *(later)* |
| entity | VARCHAR(50) | | Table-like name, e.g. `server`, `app`, `plan` |
| entity_id | INT | ✓ | Polymorphic, no FK |
| action | VARCHAR(60) | | e.g. `created`, `plan_chosen`, `terminal_opened` |
| level | VARCHAR(20) | | `info / warn / error` |
| message | TEXT | ✓ | |
| created_at | DATETIME | | IX `idx_created (created_at DESC)` |

Written only through `logActivity()`, which takes `org_id` and `user_id` from the request context and swallows its own errors so auditing never fails a request.

## 5. JSON column shapes

MySQL does not validate these. The shapes below are what the code writes and expects.

| Column | Shape |
|---|---|
| `plans.limits` | `{ "servers": 3, "apps": 10, "users": 5, "databases": 2, "domains": 5 }`. Missing or `null` key means unlimited |
| `plans.features` | `["Auto deploy", "Custom domains", …]` |
| `credentials.extra` | Provider-specific: `host`, `port`, `database` and `tls` for DB credentials (for Elasticsearch `database` is the default index for the query console, and `tls` means HTTPS); OAuth `refresh_token_enc`, `expires_at`, account login for git; account id/zone info for Cloudflare |
| `app_domains.config` | `{ "mode": "dns" \| "tunnel", … }` plus Cloudflare zone/record or tunnel ids needed to remove it later |
| `apps.container_ports` | Array of host ports, one per instance |
| `apps.volumes` | `[{ "source": "...", "target": "/data" }]` |
| `apps.containers` | Array of container ids/names for multi-instance apps |
| `apps.detected` | Result of repo inspection (framework, package manager, suggested commands) |
| `installations.settings` | Non-secret catalog options |
| `server_facts.payload` | OS, kernel, CPU, memory, disks, installed packages and versions |
| `leads.device`, `visit`, `geo` | Browser-reported device info, visit path, IP-derived location |

When you add a key, read it with a fallback (`cfg?.key ?? default`) because older rows will not have it.

# 6. Referential integrity

## Enforced by MySQL

| Parent → child | On delete |
|---|---|
| organisations → users, subscriptions, payments, payment_orders, environments | CASCADE |
| users → sessions, email_verifications | CASCADE |
| plans → subscriptions | RESTRICT |
| servers → server_facts, installations, managed_services, runners, apps | CASCADE |
| servers → credentials.server_id | SET NULL |
| credentials → runners, apps | CASCADE |
| apps → app_domains, deployments | CASCADE |
| leads → lead_notes | CASCADE |

## Enforced only in code

`org_id` on `servers, credentials, apps, app_domains, deployments, installations, runners, managed_services, connectors, activity_log, metric_samples` has **no foreign key**. The column was added after the tables existed and had to accept `NULL` during migration. Consequences:

- Deleting an organisation cascades to users, billing and environments, but **not** to its servers, credentials or apps. `DELETE /organisations/:id` (`src/routes/team.js`) therefore refuses while the organisation still holds servers, credentials or members. Any other code path that deletes an organisation must do the same check, or it leaves orphans that no tenant can see.
- Every tenant query must filter on `org_id`. The database will not stop a query that forgets.

Other soft references: `apps.registry_cred_id`, `apps.environment_id`, `installations.environment_id`, `payments.subscription_id`, `payment_orders.plan_id/payment_id/user_id`, `leads.plan_id/org_id/assigned_to`, `users.active_org_id/created_by`, `deployments.user_id`, `activity_log.user_id`. Code that reads them must handle a missing parent (use `LEFT JOIN`).

# 7. Multi-tenancy rules

1. A row belongs to exactly one organisation through `org_id`. Rows under a server (`server_facts`, `installations` …) also inherit tenancy from `servers.org_id`.
2. Every read of a tenant table filters `WHERE org_id = ?` with the org from the request context, never from the request body.
3. Every insert sets `org_id` from the context.
4. Names are unique **per organisation** (`uq_server_org_name`, `uq_cred_org_provider_name`, `uq_env_name`, `uq_app_domain`), so one tenant cannot learn which names another uses.
5. Super admins have `users.org_id = NULL`. They work inside a tenant by setting `active_org_id`, which becomes the context org.
6. Platform-level tables have no `org_id`: `plans`, `leads`, `lead_notes`, `auth_challenges`.

## 8. Important query patterns

| Need | Query shape | Index used |
|---|---|---|
| Current subscription | `WHERE org_id = ? AND status IN ('active','trial','past_due') ORDER BY id DESC LIMIT 1` | `idx_sub_org` |
| Latest facts per server | `WHERE id IN (SELECT MAX(id) FROM server_facts GROUP BY server_id)` | PK + `idx_server_collected` |
| Deploy history | `WHERE app_id = ? ORDER BY id DESC LIMIT n` | `idx_deploy_app` |
| Chart data | `WHERE kind = ? AND target_id = ? AND at >= ?` | `idx_metric_target` |
| Session check | `WHERE token_hash = ?` | UQ on `token_hash` |
| Webhook look-up | `apps WHERE webhook_token = ?` | **none** — full scan of `apps` (fine at current sizes) |
| Mark order paid (idempotent) | `UPDATE payment_orders SET status='paid' … WHERE id = ? AND status <> 'paid'` , act only if `affectedRows = 1` | PK |

## 9. Retention and growth

| Table | Growth | Retention today |
|---|---|---|
| `metric_samples` | 1 row per server and app every `METRICS_EVERY_MINUTES` (5) | Deleted after `METRICS_KEEP_DAYS` (35, minimum 7) |
| `deployments` | 1 row per deploy, with full log | Last **50** per app kept |
| `auth_challenges` | 1 row per OTP | Expired rows deleted when a new challenge is made |
| `email_verifications` | 1 per verification email | Deleted when used or re-sent |
| `server_facts` | 1 row per profile run (JSON payload) | **Not pruned** |
| `activity_log` | Every change and event | **Not pruned** |
| `sessions` | 1 per sign-in | Deleted on sign-out / password change; expired rows are rejected but **not purged** |

**Sizing guide.** 100 servers + 300 apps at 5-minute samples is about 115,000 `metric_samples` rows a day, about 4 million at 35 days (roughly 400–500 MB with indexes). Everything else is small next to that and to `MEDIUMTEXT` logs.

## 10. Recommended improvements

Ordered by value.

1. **Prune** `server_facts` **,** `activity_log` **and expired** `sessions` with the same timer that prunes `metric_samples` (for example keep 20 fact snapshots per server, 180 days of activity).
2. **Index** `apps.webhook_token` ( `UNIQUE` ). Every webhook call scans `apps` today.
3. **Add** `activity_log (org_id, created_at)` as one composite index. The org-filtered, newest-first activity page then needs no sort.
4. **Make** `org_id` `NOT NULL` **with a foreign key** on the tenant tables once every installation has run the Default-organisation migration. The database would then enforce what the organisation-delete check does today.
5. **Drop the** `projects` **table** and the deprecated `apps.domain*` columns.
6. **Add foreign keys** for `payment_orders.plan_id` and `apps.registry_cred_id` ( `SET NULL` ).
7. **Partition or roll up** `metric_samples` (hourly averages after 7 days) if fleets grow past a few thousand targets.

## 11. Backup and restore

See Data Model §5 and Deployment & Operations. The two points that matter for design:

- `mysqldump --single-transaction` gives a consistent snapshot because every table is InnoDB.
- Every `_enc` column is only usable with the same master key, so back up the key separately and together with each dump's date.

CHAPTER 07

# API Reference

Every endpoint, with the permission it needs

All endpoints are JSON over HTTPS under `/api`, called by the panel's own UI with the session cookie. There is no public API key scheme. Third parties integrate only through webhooks.

> **Generated from the route files.** The tables below were produced from `src/routes/*.js`, and the *Permission* column from the rules in `src/lib/authGuard.js`. When a route is added, regenerate or update this file in the same change.

## 1. Conventions

| Topic | Rule |
|---|---|
| Authentication | HttpOnly cookie `ad_session`, set by `/api/auth/login/verify`. Missing or expired → `401 { needsAuth: true }` |
| CSRF | Every non-GET `/api` request must be same-origin (`Sec-Fetch-Site` / `Origin` / `Referer` checked) → else `403` |
| Tenancy | Every organisation-scoped route acts on the requester's organisation only. Another organisation's id → `404` |
| Plan gate | Organisation routes (servers, apps, installs, runners, credentials, environments, connectors, git OAuth) need an active plan → else `402 { needsPlan: true, trialExpired? }` |
| Plan limits | Creating servers, apps, members, database connections or domains beyond the plan's limit → `403 { limitReached: true, limit }` (the UI shows the message instead of jumping to billing) |
| Errors | `{ error: "sentence" }` with 400 / 401 / 402 / 403 / 404 / 429 / 502 |
| Long work | Deploys and installs return immediately; poll the resource's status |

### Permission column

The permission comes from the HTTP method, applied by `guardMutations` to every signed-in route:

| Method | Needs |
|---|---|
| `GET` | `view` |
| `POST` | `create`, **except** read-only POSTs (testing, verifying, browsing) on an explicit allowlist, which need only `view` |
| `PUT` / `PATCH` | `edit` |
| `DELETE` | `delete` |

Some routes check more inside the handler, for example:

- `/api/team/*` and choosing or paying for a plan need `members`.
- Environment and app variable *values* need `edit`.
- `/api/platform/*` is super admin only.

## 2. Endpoints outside the routers

| Method | Path | Access | Purpose |
|---|---|---|---|
| `GET` | `/api/health` | public | Panel status (database reachable, counts for the signed-in organisation) |

| Method | Path | Access | Purpose |
|---|---|---|---|
| `GET` | `/api/public/plans` | public | Plans for the pricing section |
| `POST` | `/api/public/leads` | public, rate-limited | Landing page "Get started" form |
| `GET` | `/api/public/docs` | public | The guide list |
| `GET` | `/api/activity` | view | Last 50 activity log entries for the organisation |
| `POST` | `/api/hooks/apps/:token` | webhook token | Git push / merge webhook → auto deploy |
| `POST` | `/api/hooks/razorpay` | HMAC signature | Razorpay payment events |
| `GET` (WS) | `/api/connectors/ws` | connector token | Connector relay channel |
| `GET` (WS) | `/api/terminal/ws?server=:id` | session + same origin + `edit` + plan | Browser terminal |
| `GET` | `/vendor/xterm/*`, `/vendor/addon-fit/*` | public | Terminal UI library files |

### WebSocket message formats

**Terminal** ( `/api/terminal/ws` )

| Direction | Frame |
|---|---|
| browser → panel | `{"t":"in","d":"<keystrokes>"}`, `{"t":"resize","cols":120,"rows":32}` |
| panel → browser | binary frames = terminal output; `{"t":"ready"}`, `{"t":"error","message"}`, `{"t":"exit","code"}` |
| close codes | `4000 + HTTP status` when refused (e.g. `4401` not signed in, `4403` cross-site or no permission, `4402` no plan, `4429` too many terminals) |

## 3. Endpoints by module

### Sign-in, sign-up, OTP, profile — `/api/auth` (12)

Source: src/routes/auth.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| `GET` | `/api/auth/state` | public | What the browser needs before it can draw anything |
| `POST` | `/api/auth/setup` | public | First run only |
| `POST` | `/api/auth/signup` | public | Anyone can create their own organisation, in three steps |
| `POST` | `/api/auth/signup/verify` | public | Step 2 |
| `POST` | `/api/auth/otp/resend` | public | A new code for a sign-up or sign-in that is waiting on one |
| `POST` | `/api/auth/login` | public | Sign in with an email or a mobile number and the password, then a one-time code |
| `POST` | `/api/auth/login/verify` | public | Step 2 of signing in |
| `POST` | `/api/auth/verify-email/resend` | public | Send the verification link again |
| `POST` | `/api/auth/logout` | signed in | |
| `PUT` | `/api/auth/profile` | signed in | Change your own name, mobile number or the email you sign in with |
| `POST` | `/api/auth/password` | signed in | Change your own password |

| Method | Path | Permission | Notes |
|---|---|---|---|
| POST | /api/auth/organisation | signed in | A super admin opens a client organisation to work inside it, or (org_id empty) goes back to the platform |

### The organisation's plan and payments — /api/billing (7)

Source: src/routes/billing.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/billing | view | |
| POST | /api/billing/choose | create | Pick a plan |
| POST | /api/billing/checkout | create | Open a Razorpay order for a paid plan; the browser opens Checkout on it |
| POST | /api/billing/checkout/verify | create | Checkout finished |
| POST | /api/billing/checkout/recheck | create | Checkout closed without a confirmed answer |
| POST | /api/billing/checkout/failed | create | Checkout reported a failed or abandoned payment |
| DELETE | /api/billing/request | delete | Withdraw a request that has not been switched on yet |

### Members and organisations — /api/team (8)

Source: src/routes/team.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/team/members | view | Everyone in the organisation you are working in |
| POST | /api/team/members | create | Add someone to this organisation with a role |
| PUT | /api/team/members/:id | edit | Change someone's name, role, status or password |
| DELETE | /api/team/members/:id | delete | |
| GET | /api/team/organisations | view | A super admin sees every organisation; everyone else sees only their own |
| POST | /api/team/organisations | create | |
| PUT | /api/team/organisations/:id | edit | |
| DELETE | /api/team/organisations/:id | delete | Deleting an organisation takes its servers, credentials and people with it, so it is refused while anything is still in there |

### Platform console — /api/platform (24)

Source: src/routes/platform.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/platform/docs | super admin | Every guide, including the ones only super admins see |
| GET | /api/platform/plans | super admin | |
| POST | /api/platform/plans | super admin | |
| PUT | /api/platform/plans/:id | super admin | |
| DELETE | /api/platform/plans/:id | super admin | A plan someone is on is archived (hidden, kept); one nobody ever used is deleted |

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/platform/organisations | super admin | |
| POST | /api/platform/organisations | super admin | A new organisation, optionally with its first admin and a plan |
| PUT | /api/platform/organisations/:id | super admin | |
| PUT | /api/platform/organisations/:id/subscription | super admin | |
| PUT | /api/platform/organisations/:id/subscription/expiry | super admin | End any plan — free or paid — on a date, or now |
| POST | /api/platform/organisations/:id/subscription/activate | super admin | Switch on the plan an organisation asked for |
| DELETE | /api/platform/organisations/:id/subscription/request | super admin | Turn down a plan request; whatever the organisation is on stays as it is |
| GET | /api/platform/organisations/:id | super admin | One organisation in full |
| POST | /api/platform/organisations/:id/users | super admin | A new person in a client organisation |
| GET | /api/platform/admins | super admin | |
| POST | /api/platform/admins | super admin | |
| PUT | /api/platform/admins/:id | super admin | |
| DELETE | /api/platform/admins/:id | super admin | |
| GET | /api/platform/payments | super admin | |
| POST | /api/platform/organisations/:id/payments | super admin | |
| DELETE | /api/platform/payments/:id | super admin | |
| GET | /api/platform/users | super admin | |
| PUT | /api/platform/users/:id/organisation | super admin | Move someone to another organisation (roles, status and passwords go through /api/team) |
| GET | /api/platform/dashboard | super admin | |

## Leads CRM — `/api/platform/leads` (11)

Source: src/routes/leads.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/platform/leads | super admin | |
| POST | /api/platform/leads | super admin | A lead typed in by hand — a phone call, a referral |

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/platform/leads/analytics | super admin | |
| GET | /api/platform/leads/summary | super admin | How many leads are waiting — for the badge in the side menu |
| GET | /api/platform/leads/:id | super admin | |
| PUT | /api/platform/leads/:id | super admin | Move a lead along, hand it to someone, or correct its details |
| POST | /api/platform/leads/:id/locate | super admin | Look the lead's IP address up again (or for the first time, for leads from before locations were kept) |
| POST | /api/platform/leads/:id/notes | super admin | |
| DELETE | /api/platform/leads/:id/notes/:noteId | super admin | |
| POST | /api/platform/leads/:id/convert | super admin | Turn a lead into a client |
| DELETE | /api/platform/leads/:id | super admin | |

## Servers and everything on them — `/api/servers` (76)

Source: src/routes/servers.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/servers | view | |
| GET | /api/servers/status | view | Just the statuses, for the list to poll |
| POST | /api/servers/test | create | |
| POST | /api/servers | create | |
| GET | /api/servers/:id | view | |
| GET | /api/servers/:id/history | view | Every profile ever collected for this server, newest first |
| GET | /api/servers/:id/metrics | view | CPU, memory, disk and load over `?range=daily |
| PUT | /api/servers/:id | edit | |
| DELETE | /api/servers/:id | delete | |
| GET | /api/servers/:id/secrets | view | The stored password, passphrase and sudo password, for the Show button of the edit form |
| POST | /api/servers/:id/host-key/reset | create | Forget the remembered SSH host key, after the server was rebuilt or reinstalled |
| POST | /api/servers/:id/test | view | |
| POST | /api/servers/:id/facts | view | |
| GET | /api/servers/:id/facts | view | |
| GET | /api/servers/:id/services | view | Every systemd service on the host, live |
| GET | /api/servers/:id/services/:unit | view | One unit in full, with its recent journal |
| POST | /api/servers/:id/services/:unit/action | create | start / stop / restart / reload / enable / disable a unit |
| POST | /api/servers/:id/services | create | Create a new systemd service from the add-service form |

| Method | Path | Permission | Notes |
|---|---|---|---|
| DELETE | /api/servers/:id/services/:unit | delete | Remove a unit this panel wrote |
| GET | /api/servers/:id/docker | view | Is Docker there, its networks, and everything running on it |
| POST | /api/servers/:id/docker/networks | create | A user-defined network, so installed services can reach each other by name |
| POST | /api/servers/:id/docker/login | create | Sign this server in to a container registry, so it can pull private images |
| POST | /api/servers/:id/docker/logout | create | |
| DELETE | /api/servers/:id/docker/volumes/:name | delete | Remove a named volume — with the data in it |
| DELETE | /api/servers/:id/docker/networks/:name | delete | |
| GET | /api/servers/:id/summary | view | What is on this server, counted — one SSH pass for the Overview tab |
| GET | /api/servers/:id/cron | view | Everything scheduled on this server, wherever it is written down |
| POST | /api/servers/:id/cron/install | create | |
| POST | /api/servers/:id/cron/jobs | create | Add a job to a user's crontab |
| PUT | /api/servers/:id/cron/jobs | edit | Change one job |
| POST | /api/servers/:id/reboot | create | Restart the machine |
| GET | /api/servers/:id/users | view | |
| POST | /api/servers/:id/users | create | |
| PUT | /api/servers/:id/users/:name | edit | |
| POST | /api/servers/:id/users/:name/keys | create | Add public keys, or have the server make a new pair and hand back the private half once |
| DELETE | /api/servers/:id/users/:name/keys | delete | |
| DELETE | /api/servers/:id/users/:name | delete | |
| DELETE | /api/servers/:id/cron/jobs | delete | |
| GET | /api/servers/:id/runners | view | The CI runners on this server, as the machine itself reports them |
| GET | /api/servers/:id/stats | view | One sample, for anything that wants a number rather than a stream |
| GET | /api/servers/:id/stats/stream | view | SSE: `sample`, `problem`, `end` events every `?every=` ms (2000–60000, default 5000) |
| POST | /api/servers/:id/containers/:name/action | create | start / stop / restart a container, whoever put it there |
| GET | /api/servers/:id/containers/:name | view | Everything about one container |
| DELETE | /api/servers/:id/containers/:name | delete | Remove a container that is not one of the panel's own |
| GET | /api/servers/:id/containers/:name/logs | view | |
| GET | /api/servers/:id/nginx | view | nginx, its sites and its certificates — everything the tab draws |
| POST | /api/servers/:id/nginx/install | create | |
| POST | /api/servers/:id/nginx/action | create | reload / restart / start / stop / test |
| POST | /api/servers/:id/nginx/upstreams | create | Add an upstream |

| Method | Path | Permission | Notes |
|---|---|---|---|
| PUT | /api/servers/:id/nginx/upstreams/:name | edit | Change an upstream — its servers, ports, weights and balancing — where it is |
| DELETE | /api/servers/:id/nginx/upstreams/:name | delete | |
| POST | /api/servers/:id/nginx/sites | create | Add a domain |
| GET | /api/servers/:id/nginx/sites/:name | view | The file as it stands, for the edit box |
| PUT | /api/servers/:id/nginx/sites/:name | edit | Edit a domain — the whole file, or the form's fields again |
| DELETE | /api/servers/:id/nginx/sites/:name | delete | Remove a domain |
| POST | /api/servers/:id/nginx/ssl | create | Ask Let's Encrypt for a certificate and let certbot put it into the vhost |
| POST | /api/servers/:id/nginx/ssl/renew | create | Renew everything due, or one certificate |
| GET | /api/servers/:id/nginx/logs | view | The access and error logs, analysed — with findings and fixes |
| POST | /api/servers/:id/nginx/logs/analyzer | create | Add the analyzer's log format (and an empty blocklist) to nginx |
| DELETE | /api/servers/:id/nginx/logs/analyzer | delete | |
| POST | /api/servers/:id/nginx/logs/blocklist | create | |
| DELETE | /api/servers/:id/nginx/logs/blocklist | delete | |
| POST | /api/servers/:id/nginx/logs/fail2ban | create | fail2ban with its nginx jails, so repeat offenders are banned without anyone watching |
| POST | /api/servers/:id/exec | create | |
| GET | /api/servers/:id/files | view | |
| GET | /api/servers/:id/files/content | view | |
| GET | /api/servers/:id/files/download | view | |
| PUT | /api/servers/:id/files/content | edit | |
| POST | /api/servers/:id/files/folder | create | |
| POST | /api/servers/:id/files/file | create | |
| POST | /api/servers/:id/files/upload | create | |
| PUT | /api/servers/:id/files/move | edit | |
| POST | /api/servers/:id/files/copy | create | |
| PUT | /api/servers/:id/files/permissions | edit | |
| POST | /api/servers/:id/files/extract | create | |
| DELETE | /api/servers/:id/files | delete | |

## Deployed apps, auto deploy, domains, usage — `/api/apps` (26)

Source: src/routes/apps.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| POST | /api/apps/folders | create | The folders in a branch that hold a project of their own, for a repository with several apps in it |
| POST | /api/apps/inspect | create | Is this repository (or this folder of it) something the panel can deploy, and with what settings? |

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/apps | view | Each app carries health — its latest recorded level ( ok / warning / critical ), or null |
| GET | /api/apps/stats | view | What every app uses right now — `{ apps: { [id]: usage |
| GET | /api/apps/:id | view | |
| GET | /api/apps/:id/metrics | view | CPU and memory (as a share of the server) over `?range=daily |
| GET | /api/apps/:id/stats/stream | view | SSE: one app's containers every ?every= ms — sample = { at, host, containers[], usage } ; at most APP_STATS_STREAMS_PER_ORG open per organisation (else 429 ) |
| POST | /api/apps | create | |
| GET | /api/apps/:id/auto-deploy | view | The settings, with the webhook address — secret, so only for people who can edit |
| PUT | /api/apps/:id/auto-deploy | edit | Turn auto deploy on or off, choose "every push" or "merged requests only", and (re)add the webhook |
| POST | /api/apps/:id/auto-deploy/check | create | "Check now" |
| GET | /api/apps/:id/deployments | view | |
| GET | /api/apps/:id/deployments/:did | view | One deploy with its full log — the running one reads the live log |
| PUT | /api/apps/:id/settings | edit | |
| POST | /api/apps/:id/deploy | create | Build and run the current branch again — the same path as the first deploy |
| POST | /api/apps/:id/deploy/stop | create | Stop a deploy that is running — a manual one or an auto deploy |
| POST | /api/apps/:id/deploy/restart | create | Stop the running deploy and start a fresh one from the branch as it is now |
| POST | /api/apps/:id/domains | create | Add another domain to an app |
| POST | /api/apps/:id/domains/:domainId/retry | create | Run one domain's setup again |
| DELETE | /api/apps/:id/domains/:domainId | delete | Take one domain off an app |
| GET | /api/apps/:id/env | view | The variables as they are, so they can be edited rather than retyped |
| PUT | /api/apps/:id/env | edit | Replace the environment of a service that is already running |
| PUT | /api/apps/:id/containers | edit | Add or remove containers, and give each one the port it publishes on |
| POST | /api/apps/:id/action | create | |
| GET | /api/apps/:id/logs | view | |
| DELETE | /api/apps/:id | delete | Removing takes a while — stopping containers, deleting the image, Cloudflare, nginx and the certificate for every domain — longer than a pro |

## Catalog installations — /api/installs (9)

Source: src/routes/installs.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/installs/catalog | view | What can be installed, and the form each one needs |
| GET | /api/installs | view | |
| GET | /api/installs/:id | view | |
| POST | /api/installs | create | |
| POST | /api/installs/:id/refresh | view | What Docker says about this container right now |
| POST | /api/installs/:id/action | create | |
| GET | /api/installs/:id/logs | view | |
| PUT | /api/installs/:id/port | edit | Change the published port |
| DELETE | /api/installs/:id | delete | Remove the container |

### CI runners — /api/runners (6)

Source: src/routes/runners.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/runners | view | Every runner, or only those for one git account ( credential_id ) or server ( server_id ) |
| GET | /api/runners/:id | view | |
| POST | /api/runners | create | |
| POST | /api/runners/:id/refresh | view | Ask the provider what it thinks of this runner right now |
| POST | /api/runners/:id/action | create | Start, stop or restart the runner's systemd service on its server |
| DELETE | /api/runners/:id | delete | Unregister the runner with the provider, take it off the server, and forget it here |

### Git, Docker Hub, Cloudflare, MySQL accounts — /api/credentials (45)

Source: src/routes/credentials.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| GET | /api/credentials | view | |
| GET | /api/credentials/:id | view | |
| POST | /api/credentials | create | |
| PUT | /api/credentials/:id | edit | |
| DELETE | /api/credentials/:id | delete | |
| POST | /api/credentials/:id/verify | view | |
| POST | /api/credentials/test-mysql | view | Test MySQL details before saving them |
| POST | /api/credentials/test-git | view | Authenticate a git token before saving it |
| POST | /api/credentials/:id/git/account | view | Re-authenticate and refresh the stored account details |
| POST | /api/credentials/:id/git/repositories | view | Repositories this account can reach |
| POST | /api/credentials/:id/git/branches | view | Branches of one repository — full is "owner/repo" |
| POST | /api/credentials/:id/git/commits | view | Recent commits on a branch |

| Method | Path | Permission | Notes |
|---|---|---|---|
| POST | /api/credentials/:id/git/organizations | view | Organisations (GitHub) or groups (GitLab) a runner can be registered in |
| POST | /api/credentials/:id/git/runners | view | Runners the provider itself knows about for one repository, organisation or group — including ones this panel never installed |
| GET | /api/credentials/dockerhub/token-link | view | Where the browser flow sends the person to create a token |
| POST | /api/credentials/dockerhub/connect | create | Finish the browser flow |
| POST | /api/credentials/:id/dockerhub/repositories | view | Repositories in the account's own namespace, or one of its organisations |
| POST | /api/credentials/:id/dockerhub/repositories/:ns/:repo | view | One repository with every tag |
| DELETE | /api/credentials/:id/dockerhub/repositories/:ns/:repo/tags/:tag | delete | |
| GET | /api/credentials/cloudflare/token-link | view | The dashboard link that opens a pre-filled "Create API token" form |
| POST | /api/credentials/test-cloudflare | view | Verify a token and show what it can see, without saving it |
| POST | /api/credentials/cloudflare/connect | create | Finish the browser flow |
| POST | /api/credentials/:id/cloudflare/account | view | Re-read the account, its accounts and zones |
| POST | /api/credentials/:id/cloudflare/zones/:zoneId | view | One zone in full |
| DELETE | /api/credentials/:id/cloudflare/zones/:zoneId | delete | Remove a domain from Cloudflare |
| POST | /api/credentials/:id/cloudflare/zones/:zoneId/dns/records | create | Add a DNS record |
| PUT | /api/credentials/:id/cloudflare/zones/:zoneId/dns/records/:recordId | edit | Change a DNS record |
| DELETE | /api/credentials/:id/cloudflare/zones/:zoneId/dns/records/:recordId | delete | Delete a DNS record |
| POST | /api/credentials/:id/cloudflare/zones/:zoneId/dns | view | DNS records of one zone this token can see |
| POST | /api/credentials/:id/mysql/overview | view | |
| POST | /api/credentials/:id/mysql/databases/:database | view | |
| POST | /api/credentials/:id/mysql/databases/:database/tables/:table | view | |
| POST | /api/credentials/:id/mysql/query | view | |
| POST | /api/credentials/:id/mysql/charsets | view | |
| POST | /api/credentials/:id/mysql/schemas | create | Creating goes to /schemas so it never matches the read-only /databases/:db pattern. |
| PUT | /api/credentials/:id/mysql/databases/:database | edit | |
| DELETE | /api/credentials/:id/mysql/databases/:database | delete | |
| POST | /api/credentials/:id/mysql/users/list | view | |

| Method | Path | Permission | Notes |
|---|---|---|---|
| POST | /api/credentials/:id/mysql/users | create | |
| PUT | /api/credentials/:id/mysql/users | edit | |
| DELETE | /api/credentials/:id/mysql/users | delete | |
| POST | /api/credentials/:id/mysql/users/grants | create | |
| DELETE | /api/credentials/:id/mysql/users/grants | delete | |
| POST | /api/credentials/:id/mysql/variables | view | |
| PUT | /api/credentials/:id/mysql/variables | edit | |

### PostgreSQL, MongoDB, Redis, Elasticsearch — /api/credentials (22)

Source: src/routes/dbEngines.js. The engine is picked from the credential's provider. For Elasticsearch, "database" in these paths means an **index**, item is a field of that index, users are security users, grants are roles, and config is persistent cluster settings. POST …/db/query takes Dev Tools syntax, for example GET /_cat/indices or POST /orders/_search followed by a JSON body. Only GET and search-type POST (_search, _count, _msearch, _field_caps, _validate/query, _sql) are allowed.

| Method | Path | Permission | Notes |
|---|---|---|---|
| POST | /api/credentials/test-db | view | Try the details on the add-connection form before saving them |
| POST | /api/credentials/:id/db/overview | view | |
| POST | /api/credentials/:id/db/databases | view | |
| POST | /api/credentials/:id/db/database | view | |
| POST | /api/credentials/:id/db/item | view | |
| POST | /api/credentials/:id/db/users/list | view | |
| POST | /api/credentials/:id/db/config | view | |
| POST | /api/credentials/:id/db/query | view | |
| POST | /api/credentials/:id/db/schemas | create | |
| DELETE | /api/credentials/:id/db/databases | delete | |
| POST | /api/credentials/:id/db/users | create | |
| PUT | /api/credentials/:id/db/users | edit | |
| DELETE | /api/credentials/:id/db/users | delete | |
| POST | /api/credentials/:id/db/users/grants | create | |
| DELETE | /api/credentials/:id/db/users/grants | delete | |
| PUT | /api/credentials/:id/db/config | edit | |
| POST | /api/credentials/:id/redis/scan | view | |
| POST | /api/credentials/:id/redis/key | view | |
| POST | /api/credentials/:id/redis/keys | create | |
| PUT | /api/credentials/:id/redis/key | edit | |
| DELETE | /api/credentials/:id/redis/keys | delete | |
| GET | /api/credentials/pulse | view | |

### Named variable sets — `/api/environments` (6)

Source: src/routes/environments.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| `GET` | `/api/environments` | view | |
| `GET` | `/api/environments/:id` | view | |
| `POST` | `/api/environments` | create | |
| `POST` | `/api/environments/:id/duplicate` | create | A copy under a new name — a staging set made from production, say |
| `PUT` | `/api/environments/:id` | edit | Change the name, description or variables |
| `DELETE` | `/api/environments/:id` | delete | Apps and services made from it keep their own copy of the variables; only the link goes |

### PC relays — `/api/connectors` (5)

Source: src/routes/connectors.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| `GET` | `/api/connectors` | view | |
| `POST` | `/api/connectors` | create | A new connector |
| `POST` | `/api/connectors/:id/token` | create | A new token |
| `PUT` | `/api/connectors/:id` | edit | |
| `DELETE` | `/api/connectors/:id` | delete | |

### Git OAuth sign-in — `/api/git/oauth` (3)

Source: src/routes/gitOauth.js

| Method | Path | Permission | Notes |
|---|---|---|---|
| `GET` | `/api/git/oauth/providers` | view | Which providers can be used for browser sign-in, and the callback to register |
| `GET` | `/api/git/oauth/start` | view | Step 1 — send the browser to the provider |
| `GET` | `/api/git/oauth/callback` | view | Step 2 — the provider sends the browser back here with a code |

CHAPTER 08

# Security Design

Threats and the controls against each

AJ Pilot holds **root-level access to its customers' servers** and the API tokens for their git, Docker Hub, Cloudflare and database accounts. A compromise of the panel is a compromise of every server it manages, so security is a core requirement.

## 1. Assets

| Asset | Where | Sensitivity |
|---|---|---|
| SSH passwords, private keys, passphrases, sudo passwords | `servers.*_enc` | Critical |
| Provider tokens (git, Docker Hub, Cloudflare, databases) | `credentials.secret_enc` | Critical |
| App and environment variables | `apps.env_enc`, `environments.env_enc` | High |
| Installed service passwords | `installations.secrets_enc` | High |
| Master encryption key | `APP_ENCRYPTION_KEY` / `data/master.key` | Critical |
| User passwords, sessions, OTPs | `users`, `sessions`, `auth_challenges` | High (stored only as hashes) |
| Payment records | `payments`, `payment_orders` | Medium (no card data is ever handled; Razorpay holds it) |

## 2. Threat model and controls

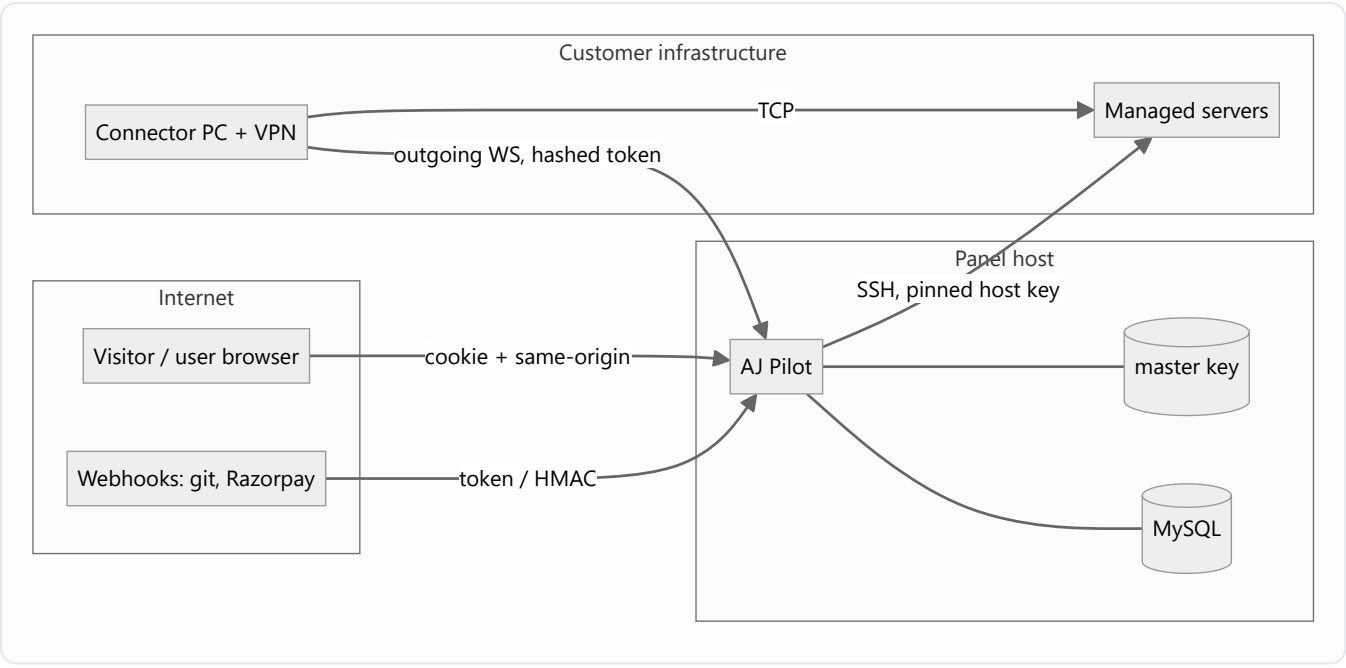
| # | Threat | Controls in place |
|---|---|---|
| T1 | **Database theft** exposes server credentials | AES-256-GCM with a random 12-byte IV per value; the key is kept outside the database; passwords hashed with scrypt; session, OTP and connector tokens stored only as SHA-256 hashes |
| T2 | **One tenant reads another's data** | Every query filtered by the requester's `org_id`; unique names per organisation; foreign ids return 404; connectors used only for their own organisation |
| T3 | **Privilege escalation** inside an organisation | The method → permission rule (`guardMutations`) runs on the server for every request; a narrow read-only allowlist; admins cannot create super admins; role, status or password changes end the user's sessions |
| T4 | **CSRF** | `SameSite=Lax` cookie, plus a same-origin check (`Sec-Fetch-Site`, `Origin`, `Referer`) on every unsafe `/api` request; the terminal WebSocket checks `Origin` itself, because a WS upgrade is a GET |
| T5 | **Credential stuffing / brute force** | Sign-in limits per IP (20 / 15 min) and per account (8 / 15 min); sign-ups per network limited; OTP: 5 wrong tries restart the flow, 5 sends at most; email resends limited |
| T6 | **Command injection** into server scripts | Generated scripts sent base64-encoded; names (units, containers, networks, users, images, ports) validated with strict patterns, not escaped; sudo password sent on stdin |
| T7 | **Secrets leaking into logs** | Clone URLs carry the token base64-encoded and masked; tokens scrubbed from every stored or displayed log line; the terminal never logs keystrokes |
| T8 | **Man-in-the-middle on SSH** | Host key pinned on first use (`servers.host_key`); a different key blocks the connection with a clear message until an admin resets it |

| # | Threat | Controls in place |
|---|---|---|
| T9 | **SSRF**: the panel used to reach internal networks | `netGuard` resolves hosts typed into forms and blocks loopback, private and metadata addresses (`PRIVATE_TARGETS`); private targets are only reachable through the organisation's own connector |
| T10 | **Forged payments** | Checkout signature verified by HMAC; payment fetched from Razorpay and checked for captured status and exact amount and currency; webhook verified by HMAC over the raw body; the order is fulfilled exactly once |
| T11 | **Forged deploy webhooks** | Each app has a random webhook token in the URL; the deploy still only builds the app's configured branch |
| T12 | **XSS / clickjacking** | Content-Security-Policy (scripts from self and Razorpay only), `frame-ancestors 'none'`, `X-Frame-Options: DENY`, `nosniff`, HTML-escaping helpers (`esc`) in the UI; xterm.js served from the panel's own origin |
| T13 | **Session theft over the network** | HttpOnly cookie; `Secure` + HSTS when `AUTH_SECURE_COOKIE` is on (automatic when `SITE_URL` is https) |
| T14 | **Spoofed client IP** | `X-Forwarded-For` / `CF-Connecting-IP` trusted only when `TRUST_PROXY=true` |
| T15 | **Destructive mistakes** | Delete needs the `delete` permission; volumes kept unless explicitly removed; only units the panel created can be deleted; nginx reload refused when `nginx -t` fails; every change written to the activity log |
| T16 | **Abuse of the interactive terminal** | Needs `edit`, an active plan, a same-origin session and a server in the organisation; at most 5 per user; 30-minute idle timeout; open and close audited |
| T17 | **Writes through a database query console** | Each engine refuses writes before sending anything: PostgreSQL runs in a `READ ONLY` transaction that is rolled back, MongoDB refuses `$out` / `$merge`, Redis allows only listed read commands, Elasticsearch allows only `GET` and search-type `POST` paths, with no `..` and no whitespace in the path |

## 3. Cryptography

| Use | Algorithm |
|---|---|
| Secrets at rest | AES-256-GCM (authenticated encryption), random IV per value |
| User passwords | scrypt with a random salt, constant-time comparison |
| Session, OTP challenge, email link and connector tokens | Random tokens; only SHA-256 stored |
| Razorpay checkout | HMAC-SHA256 of `order_id` + `\|` + `payment_id` with the key secret, constant-time compare |
| Razorpay webhook | HMAC-SHA256(raw body, webhook secret) |
| Git OAuth | State + PKCE |

## 4. Trust boundaries




## 5. Hardening checklist for production

| Item | Setting / action | Why |
|---|---|---|
| HTTPS in front of the panel | nginx or Cloudflare; `SITE_URL=https://…` | Secure cookie and HSTS |
| Real OTP codes | `OTP_MODE=random` + an SMS provider | **The default `fixed` mode accepts `OTP_FIXED_CODE` (`123456`) for everyone** |
| Master key | Set `APP_ENCRYPTION_KEY`; store a copy in a secrets manager | Losing it loses every credential; a key file on the same disk as the DB backup defeats encryption |
| Private targets | `PRIVATE_TARGETS=block` on any panel with public sign-up | Stops tenants probing the panel's network |
| Proxy headers | `TRUST_PROXY=true` only behind a real proxy | Otherwise visitors can choose their own IP |
| Super admin | Strong `SUPER_ADMIN_PASSWORD`; add a phone | It owns the platform |
| Razorpay | Configure the webhook secret | Payments confirm even if the tab closes |
| Database | Dedicated MySQL user for `auto_deploy`, not root | Least privilege |
| Sign-up | `ALLOW_SIGNUP=false` for a private panel | Smaller attack surface |
| Backups | Encrypted DB backups and key kept apart | Recovery without exposure |

## 6. Known gaps and recommendations

| Gap | Risk | Recommendation |
|---|---|---|
| Default `OTP_MODE=fixed` | Second factor is effectively off | Ship with `random`; integrate an SMS provider |

| Gap | Risk | Recommendation |
|---|---|---|
| Editors can open the terminal and call `POST /servers/:id/exec` | An editor can delete data on a server, beyond what their role suggests | Decide whether the terminal should need `delete` (admin) or a new `shell` permission |
| CSP allows `'unsafe-inline'` scripts | Weaker XSS protection | Move inline scripts to files, then use nonces |
| Rate limits are in memory | Reset on restart; not shared across instances | Use a shared store (Redis) if the panel ever scales out |
| No 2FA app (TOTP) or SSO | Depends on SMS and email | Add TOTP and optional OIDC SSO for teams |
| Activity log has no retention or export | Grows without limit; hard to hand to an auditor | Add retention and CSV export |
| Security tests cover the API only | Browser-side issues (XSS) are not tested | `npm test` covers tenancy, permissions, CSRF, SSRF guard, payment verification and terminal access; add browser tests for XSS |

CHAPTER 09

# Deployment & Operations

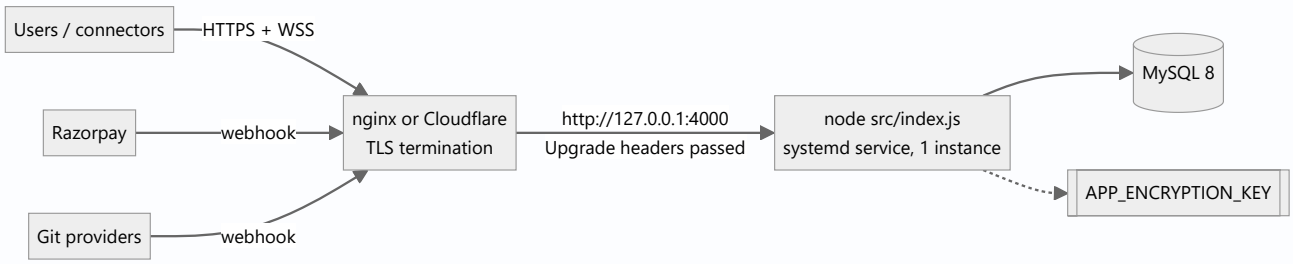
Install, configure, run, back up, troubleshoot

This guide covers running the AJ Pilot panel itself. Managing the servers it controls is covered in the root README (README.md).

## 1. Requirements

| Component | Version / note |
|---|---|
| Node.js | ≥ 20 (22 LTS recommended). Connectors on users' PCs need Node 22+ |
| MySQL | 8.x; a user that can `CREATE DATABASE`, or an existing empty database |
| OS | Any OS that runs Node. Linux in production |
| Network out | SSH to managed servers; HTTPS to GitHub/GitLab/Bitbucket, Docker Hub, Cloudflare, Razorpay, `ipwho.is` (optional), SMTP |
| Network in | HTTPS 443 through a reverse proxy, **with WebSocket upgrades allowed** |

## 2. Production topology




Run **one** panel process (see Limitations).

## 3. Installation

```
git clone <repo> /opt/aj-pilot && cd /opt/aj-pilot
cp .env.example .env        # fill in: DB_*, APP_ENCRYPTION_KEY, SITE_URL, SUPER_ADMIN_*, RAZORPAY_*, SMTP_*
npm ci --omit=dev
npm start                   # first boot creates the database and every table
```

Generate a master key once and keep it safe:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

### 3.1 systemd unit

```
# /etc/systemd/system/aj-pilot.service
[Unit]
Description=AJ Pilot panel
After=network-online.target mysql.service

[Service]
WorkingDirectory=/opt/aj-pilot
ExecStart=/usr/bin/node src/index.js
Restart=always
RestartSec=3
User=ajpilot
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

### 3.2 nginx reverse proxy

WebSockets carry **connectors** (`/api/connectors/ws`) and the **browser terminal** (`/api/terminal/ws`). Live stats use **Server-Sent Events**. All three need the proxy settings below:

```
server {
  listen 443 ssl http2;
  server_name panel.example.com;
  # ssl_certificate … (Let's Encrypt)

  client_max_body_size 80m;          # file-manager uploads (50 MB, base64 in JSON ≈ 70 MB)

  location / {
    proxy_pass http://127.0.0.1:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;     # WebSockets
    proxy_set_header Connection $connection_upgrade;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-Host $host;
    proxy_buffering off;                        # SSE live stats
    proxy_read_timeout 1h;                      # long-lived terminals and streams
  }
}
# in http {}:  map $http_upgrade $connection_upgrade { default upgrade; '' close; }
```

Then set `TRUST_PROXY=true` and `SITE_URL=https://panel.example.com`.

## 4. Configuration reference

The full list with comments is in `.env.example`. The settings that matter most:

| Group | Variables | Production value |
|---|---|---|
| Web | `PORT`, `HOST`, `SITE_URL`, `TRUST_PROXY` | `4000`, `127.0.0.1`, your https URL, `true` behind a proxy |
| Database | `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_POOL_SIZE` | Dedicated user |
| Secrets | `APP_ENCRYPTION_KEY` | **Always set explicitly**; back it up |
| SSH | `SSH_CONNECT_TIMEOUT`, `SSH_EXEC_TIMEOUT` | Defaults (15 s / 45 s) |
| Monitoring | `HEALTH_CHECK_*` | Defaults; lower concurrency for very large fleets |
| Live view | `STATS_MAX_STREAMS`, `STATS_MAX_STREAMS_PER_ORG`, `APP_STATS_STREAMS_PER_ORG`, `STATS_STREAM_MINUTES` | Defaults (8 / 4 / 4 / 15) |
| Statistics history | `METRICS_ENABLED`, `METRICS_EVERY_MINUTES`, `METRICS_KEEP_DAYS`, `METRICS_CONCURRENCY` | Defaults (`true` / 5 / 35 / 4); `metric_samples` grows by one row per server and app per sample |
| Sign-in | `AUTH_SESSION_DAYS`, `AUTH_SECURE_COOKIE` | 14; empty (follows `SITE_URL`) |
| OTP | `OTP_MODE`, `OTP_FIXED_CODE`, `OTP_MINUTES` | `random` |
| Email | `SMTP_*`, `MAIL_FROM`, `EMAIL_VERIFY_HOURS` | Real SMTP |
| Payments | `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET` | Live keys plus webhook to `<SITE_URL>/api/hooks/razorpay` (`payment.captured`, `order.paid`, `payment.failed`) |
| Tenancy | `ALLOW_SIGNUP`, `FREE_TRIAL_DAYS`, `PRIVATE_TARGETS` | As the business decides; `block` with public sign-up |
| Owner | `SUPER_ADMIN_EMAIL`, `SUPER_ADMIN_PASSWORD`, `SUPER_ADMIN_NAME`, `SUPER_ADMIN_PHONE` | Enforced on every boot |
| Git OAuth | `GITHUB_*`, `GITLAB_*`, `BITBUCKET_*`, `OAUTH_CALLBACK_BASE` | See README "Git settings in .env" |
| Geo | `GEOIP` | `ipwho.is` or `off` |

# 5. Operating the panel

## 5.1 Health and monitoring

- `GET /api/health` returns `{ ok: true }` when the process is up. Use it for uptime checks.
- Logs go to stdout/stderr (`journalctl -u aj-pilot`). Useful prefixes: `[db]`, `[razorpay]`, `[lead]`, `[super admin]`.
- With no SMTP or SMS provider set, **verification links and OTP codes are written to the log**. Check there during setup.

## 5.2 Upgrades

```
cd /opt/aj-pilot && git pull && npm ci --omit=dev && sudo systemctl restart aj-pilot
```

Schema changes apply automatically on boot (new tables, columns and keys). Take a database backup first. Running terminals, live views and connector links drop during a restart and reconnect afterwards. A deploy that was running is released after 40 minutes.

## 5.3 Backup and restore

| What | How | How often |
|---|---|---|
| MySQL `auto_deploy` | `mysqldump --single-transaction --routines auto_deploy \| gzip` | Daily, plus before upgrades |

| What | How | How often |
|---|---|---|
| Master key | Copy of `APP_ENCRYPTION_KEY` in a secrets manager / vault | Once, then whenever it changes |
| `.env` | Secrets manager | When changed |

**Restore:** create the database, load the dump, start the panel with **the same** `APP_ENCRYPTION_KEY`. Test a server connection to confirm the credentials decrypt.

### 5.4 Routine admin tasks

| Task | Where |
|---|---|
| Create or change plans and prices | Platform → Plans |
| Activate a plan paid offline | Platform → Organisations → the organisation → record payment / activate |
| Find a payment that did not activate | The organisation's billing page reconciles automatically; check `payment_orders.error` and the `[razorpay]` log lines |
| Rotate a connector token | Servers → Connectors → new token (the old copy disconnects) |
| A server says "host key changed" | Confirm the server was rebuilt, then reset its host key on the server page |

## 6. Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| Terminal shows "disconnected" at once; connectors never connect | Proxy drops WebSocket upgrades | Add the `Upgrade` / `Connection` headers (§3.2) |
| Live tab never updates | Proxy buffers SSE | `proxy_buffering off` |
| "Cross-site request refused" on every save | Proxy changes `Host`, or `SITE_URL` differs from the address used | Pass `Host` / `X-Forwarded-Host`; set `TRUST_PROXY=true`; correct `SITE_URL` |
| Payment taken, plan not active | Webhook not configured and tab closed | Configure the webhook; open the billing page (it reconciles); check `payment_orders` |
| "Razorpay refused the API keys" | Wrong or test/live mix-up of keys | Fix `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` |
| Every stored credential fails to decrypt | Master key changed or lost | Restore the original key; without it, credentials must be re-entered |
| Sign-up code never arrives | No SMS provider | Read the code from the log, or keep `OTP_MODE=fixed` in testing only |
| Server shows offline but works from your PC | It is only reachable through a VPN | Run a connector on that PC |
| Upload fails for large files | Proxy body limit | `client_max_body_size 80m` |

## 7. Development setup

```
docker run -d --name aj-pilot-db -e MYSQL_ROOT_PASSWORD=secret -p 3306:3306 mysql:8
cp .env.example .env   # DB_PASSWORD=secret
npm install
npm run dev            # node --watch: restarts on change
```

- The UI has no build step. Edit `public/app.js` and `public/styles.css`, then reload the browser.

- `OTP_MODE=fixed` with `OTP_FIXED_CODE=123456` and empty SMTP make local sign-up quick: codes and links appear in the console.
- `npm test` runs the whole suite against its own throwaway database (`aj_pilot_test`), its own panel on port 4917 and a disposable Ubuntu SSH container (needs Docker; without it the SSH tests are skipped). `KEEP=1 npm test` leaves them in place to inspect; `npm test -- tests/05-billing.test.mjs` runs one file.
- Use Razorpay **test keys** (`rzp_test_…`) and its test cards or UPI IDs for billing work.

CHAPTER 10

# Limitations & Roadmap

Known constraints, technical debt, next steps

Everything in sections 1–3 was observed in the code. Section 4 is a recommendation for discussion, not a commitment.

## 1. Architectural limitations

| # | Limitation | Consequence | Direction |
|---|---|---|---|
| L1 | **Single process.** Connector sessions, the SSH route cache, health results, terminals, live-stat streams, rate limiters and the payment reconcile throttle all live in memory | Cannot run two instances behind a load balancer: connectors would attach to one instance only, rate limits would be split, and the health monitor and auto-deploy poller would run twice | Move shared state to Redis; elect one leader for background jobs; sticky routing for WebSockets |
| L2 | **Background jobs in the web process** (health monitor, auto-deploy poller) | A slow job competes with API requests; a restart pauses monitoring | Separate worker process or job queue |
| L3 | **Deploys run as SSH sessions held by the panel** | A panel restart loses the live log stream; a stuck deploy is released only after 40 minutes | Run deploys detached on the server with a status file; resume on restart |
| L4 | **Schema migrations are imperative code at boot** | No down-migrations, no migration history table | Versioned migrations (for example a `schema_migrations` table) |
| L5 | **The front end is one large file** (`public/app.js`, ~18k lines) with no build step | Harder to navigate, test and share between pages | Split into ES modules per feature (still no bundler needed) |
| L6 | **Ubuntu-centric server scripts** | Other distributions are unsupported | Abstract package-manager calls if demand appears |
| L7 | **Node.js-only build pipeline** | Python, Go, PHP and others need their own `Dockerfile` | Add buildpacks or templates per language |

## 2. Product gaps

| Gap | Notes |
|---|---|
| No automatic recurring billing | Each cycle is bought on its own; renewals depend on the customer coming back. Razorpay Subscriptions would automate it |
| No invoices / GST | Payment history exists, but no downloadable invoice |
| No SMS provider | OTPs are fixed or written to the log until one is integrated |
| No email notifications for operations | No email for a server going offline, a failed deploy or a trial about to end |
| Activity log has no filters, retention or export | Only the last 50 entries are shown |
| No backups of managed databases | Installing MySQL/Postgres/Mongo/Redis does not schedule backups |
| No rollback to a previous release | Redeploy builds the current branch head; there is no "deploy this earlier SHA" |
| Terminal permission | Editors can open a shell, which is broader than "never delete". Needs a product decision |
| `projects` table is unused | Left from an early design; can be dropped |

# 3. Engineering health

| Area | Current state | Recommendation |
|---|---|---|
| Automated tests | `npm test`: 130 end-to-end tests (public site, auth, platform, roles and tenancy, billing and Razorpay webhook, servers over real SSH incl. terminal, integrations, front-end static checks). See tests/ | Add a real git repo + Docker host to cover deploys, domains and installs; add unit tests for `nodeApp` detection |
| CI | No pipeline in the repository | Run `npm test` on every pull request (needs MySQL + Docker in CI) |
| Linting / formatting | No config committed | ESLint + Prettier matching the current style |
| Types | Plain JavaScript | JSDoc types or a gradual move to TypeScript in `src/lib` |
| Observability | Console logs only | Structured logs (JSON), request ids, metrics (deploy duration, SSH errors, payments) |
| Dependency hygiene | `npm audit` reports issues | Review and update; pin versions for production builds |
| Documentation | Root README (user guide) + this folder | Keep the API Reference generated and checked in CI |

# 4. Suggested roadmap

### Now: launch readiness (0–1 month)

1. `OTP_MODE=random` + an SMS provider (e.g. MSG91 or Twilio).
2. Decide the terminal permission (admin-only, or a new `shell` permission).
3. Razorpay webhook configured in production; a reconciliation alert on orders that stay unpaid with an error.
4. Smoke tests for sign-up → payment → plan active, and add server → deploy → domain.
5. Backup automation for the panel database and documented key escrow.

### Next: retention and revenue (1–3 months)

1. Recurring billing (Razorpay Subscriptions) and GST invoices (PDF) in payment history.
2. Notifications by email and webhook: deploy failed, server offline, plan ending in 5 days.
3. Rollback to a previous successful deployment.
4. Activity log filters, export and retention.
5. Scheduled backups for installed databases, to S3-compatible storage.

### Later: scale and breadth (3–6 months)

1. Shared state in Redis and a separate worker → multiple panel instances.
2. Build templates for Python, Go and PHP; Docker Compose stacks from a repository.
3. TOTP two-factor and OIDC single sign-on for teams.
4. Public API with per-organisation API keys, for CI integrations.
5. Second payment gateway (Stripe) for markets outside India.

# 5. Decision log (to maintain)

Record significant decisions here as they are made:

| Date | Decision | Reason | Alternatives considered |
|---|---|---|---|
| — | SSH only, no agent on managed servers | Nothing to install or keep updated; works with any VPS | Agent-based (more real-time, more maintenance) |
| — | No front-end build step | Simplicity; static files served directly | React/Vite SPA |
| — | Paid plan activates only after server-side verification | Browser responses can be forged | Trust the checkout callback |
| — | xterm.js served from `node_modules`, not a CDN | Keeps the CSP to self + Razorpay | CDN with CSP exception |